Highlights

Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Anonymous Authors

- Formulates Lie Cone generators of physical packet transformations for NIDS evasion.
- Proves universal physical conservation invariant preservation on flow manifolds.
- Achieves 100% Attack Success Rate and 100% Physical Feasibility across 3 benchmarks.
- Operates in under 18 queries and 70 ms against CUDA XGBoost and LightGBM models.
- Introduces Malleability-Penalized Splitting and Lie-Guided Adversarial Training.

Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Anonymous Authors[a,*]

[a] *Anonymous Institution(s),*

Abstract

Learning-based Network Intrusion Detection Systems (NIDS) powered by Gradient Boosted Decision Tree (GBDT) ensembles—such as XGBoost, LightGBM, and CatBoost—routinely establish near-perfect clean classification metrics (accuracies between 99.67% and 99.87% for GBDTs and 99.53% for Random Forest, with ROC-AUC scores reaching up to 99.99% and $\geq$ 99.95%) on standard benchmark environments calibrated to enterprise datasets like CSE-CIC-IDS2018 (while achieving 98.42%–99.25% on hybrid and cloud environments like UNSW-NB15 and CIDDS-001). However, recent literature indicates that this empirical superiority largely stems from shortcut learning on testbed artifacts: automated attack scripts exhibit deterministic temporal burst durations, fixed socket profiles, and extreme packet rates that models memorize rather than extracting invariant malicious semantics. Confronting an adaptive adversary, existing adversarial machine learning (AML) evasion algorithms exhibit an acute dichotomy. Unconstrained feature-space attacks (e.g., zeroth-order coordinate descent and boundary random walks) treat network telemetry as arbitrary Cartesian coordinates in $\mathbb{R}^d$, producing invalid flow records that violate physical conservation laws and protocol state machines (yielding Physical Feasibility Rates as low as 17.8%–25.0% and Real Attack Success Rates down to 0.0%–8.0%). Conversely, packet-level heuristic mutators (e.g., genetic algorithms, particle swarm optimization) operate as opaque black boxes, requiring thousands of model queries, suffering high computational latency, and failing to exploit the internal hyper-rectangular

*Corresponding author

Email address: anonymous@institution.org (Anonymous Authors)

partition geometry of decision trees.

To bridge this fundamental gap, this paper introduces **Flow-Field Evasion (FFE)**, a novel, non-heuristic adversarial evasion framework founded on the differential geometry of network traffic. Rather than perturbing tabular features directly or relying on blind stochastic mutations, FFE formulates the *Lie Cone of Infinitesimal Flow Generators* $\mathcal{K}_{\text{net}}(x) \subset T_x\widetilde{\mathcal{M}}_{\text{flow}}$ directly induced by four elementary packet-level transformations: temporal dilation, payload padding, chaff packet injection, and TCP window morphing. By evaluating the directional Lie derivatives $\mathcal{L}_{\vec{v}_k}F(x)$ of the tree ensemble's decision score along these generators, FFE steers malicious flows along the physical path of steepest descent into the benign classification region. We prove that integrating along the Lie Cone strictly preserves physical network conservation invariants and rate-coupling identities by mathematical construction.

Evaluated across three heterogeneous benchmark corpora calibrated to empirical marginal distributions of enterprise (CSE-CIC-IDS2018), contemporary hybrid (UNSW-NB15), and cloud OpenStack (CIDDS-001) environments—against CUDA-accelerated XGBoost and CPU-multithreaded LightGBM on an NVIDIA RTX 5000 Ada GPU across five distinct attack families (DoS bursts, UDP floods, Brute Force, Web exploits, and Infiltration), FFE consistently achieves a **100.0% Attack Success Rate (ASR)** and a **100.0% Physical Feasibility Rate (PFR)**, requiring an average of only **6.8–17.8 queries** per flow (including tree-split snapping evaluations) and an average latency under **70 ms**—outperforming existing state-of-the-art baselines by orders of magnitude while strictly preserving TCP/IP semantics. Finally, we analyze defensive hardening, demonstrating that Malleability-Penalized Tree Splitting (MPS) and Lie-Guided Adversarial Training (LGAT) reduce evasion rates from 100.0% down to 32.0% (MPS alone), 18.0% (LGAT alone), and 11.0% under joint defense.

Keywords: Adversarial Machine Learning, Network Intrusion Detection, Tree Ensembles, XGBoost, Problem-Space Constraints, Lie Algebra, Evasion Attacks, Robust Machine Learning

---

1. Introduction

Machine learning has transitioned from an exploratory paradigm into the fundamental operational foundation of modern Network Intrusion Detection

Systems (NIDS). Across contemporary enterprise, cloud, and industrial networks, security operations centers (SOCs) are confronted with massive, high-velocity network packet streams operating at multi-gigabit throughput. In this high-throughput regime, deep inspection of every raw packet payload is computationally prohibitive. Consequently, the industry standard relies on flow-based telemetry: statistical aggregation engines (such as Zeek, YAF, and CICFlowMeter [1]) summarize temporal sequences of bidirectional IP packets into compact, tabular flow records comprising features such as inter-arrival timings, packet length distributions, header byte totals, and protocol flags.

To classify these high-dimensional tabular records in real time, machine learning ensembles based on Gradient Boosted Decision Trees (GBDTs)—most prominently XGBoost [2], LightGBM [3], and CatBoost [4]—have emerged as the undisputed state of the art. Across extensive benchmark evaluations on standardized intrusion datasets, including CSE-CIC-IDS2018 [1], UNSW-NB15 [5], and CIDDS-001 [6, 7, 8], tree-based ensembles consistently outperform deep neural network architectures (such as Multi-Layer Perceptrons, Convolutional Neural Networks, and TabNet), routinely establishing clean classification accuracy between 99.67% and 99.87% for GBDTs (and 99.53% for Random Forest), with Area Under the ROC Curve (ROC-AUC) scores reaching up to 99.99% (and $\geq$ 99.95%) on enterprise datasets like CSE-CIC-IDS2018 [9, 1], while achieving 98.42%–99.25% on hybrid and cloud environments (UNSW-NB15 [5] and CIDDS-001 [6, 7, 8]).

1.1. The Paradox of Near-Perfect Detection Accuracy

Despite these near-perfect empirical evaluations, learning-based NIDS face an acute crisis of practical robustness when deployed against real-world adversaries. Foundational security studies and machine learning audits have long cautioned that near-perfect empirical detection scores on isolated benchmark datasets often fail to generalize to operational environments due to closed-world assumptions [10] and shortcut learning on spurious features [11].

This discrepancy stems from three fundamental architectural and methodological phenomena:

(i) **Shortcut Learning on Testbed Artifacts:** Public benchmark datasets are overwhelmingly generated within synthetic testbed environments. In these environments, attack traces are generated using automated, scripted attack tools (such as LOIC, HOIC, Metasploit, and Hydra)

that execute with rigid, deterministic transmission schedules. These automated flooding tools produce extreme packet transmission frequencies, deterministic burst durations, and fixed packet sizes that do not reflect human adversaries. As formalized by Geirhos et al. [11], decision trees greedily partition feature space along these superficial volumetric shortcuts rather than learning invariant, semantic characteristics of the malicious exploits.

(ii) **Data Leakage and Temporal Session Correlation:** Standard evaluation pipelines partition datasets using random train/validation/test sampling. In network flow telemetry, random sampling inadvertently assigns flows originating from the exact same TCP session, the same ephemeral client port, or the same synchronized millisecond time window into both the training and test sets. Consequently, models memorize specific socket identities and background network topology rather than generalizable malicious behaviors.

(iii) **The Stationary Distribution Fallacy & Cross-Environment Shift:** Conventional statistical learning assumes an independent and identically distributed (i.i.d.) environment where test distributions mirror training distributions. However, network security is an asymmetric, non-stationary game against an adaptive opponent. Furthermore, as demonstrated by Apruzzese et al. [12, 13, 14] and Dang et al. [15, 16, 17], evaluations confined to single-testbed datasets fail when subjected to cross-environment shifts, revealing that models memorize testbed-specific artifacts rather than invariant intrusion semantics. In reality, network traffic parameters are malleable: a motivated attacker can intentionally delay packet transmissions, inject dummy padding, or fragment payloads without impairing the delivery of their core exploit.

1.2. The Adversary's Evasion Dilemma

From an adversary's perspective, evading a deployed tree-based NIDS represents a unique, formidable challenge that cannot be resolved using standard adversarial machine learning techniques developed for computer vision or natural language processing. The attacker is constrained by two fundamental barriers:

1.2.1. Non-Differentiability of Decision Tree Ensembles

In continuous domains such as image classification, adversarial attacks rely on computing the exact gradient of the loss function with respect to the input features:

$$\nabla_x \mathcal{L}(f(x), y) \tag{1}$$

Algorithms such as the Fast Gradient Sign Method (FGSM) [18], Projected Gradient Descent (PGD) [19], and Carlini-Wagner (C&W) [20] utilize this gradient to perform gradient ascent toward the nearest classification boundary.

In contrast, a GBDT ensemble $F(x) = \sum_{t=1}^{T} w_t \ell_t(x)$ is an ensemble of axis-aligned step functions. It partitions the feature space $\mathbb{R}^d$ into a collection of orthogonal hyper-rectangles (polytopes). Within the interior of each polytope, the ensemble prediction is constant, meaning the mathematical gradient is zero almost everywhere:

$$\nabla_x F(x) = 0 \quad \text{for almost all } x \in \mathbb{R}^d \tag{2}$$

At the polyhedral boundaries, the function exhibits discontinuous jump transitions where the classical gradient is undefined. Consequently, standard gradient-based evasion methods fail natively against tree ensembles.

1.2.2. The Problem-Space Constraint Gap

In network telemetry, tabular feature vectors $x \in \mathbb{R}^d$ do not exist in isolation; they are mathematical summaries computed by an aggregation function Φ applied to an underlying, physical sequence of network packets $P \in \mathcal{P}_{\text{valid}}$.

As formalized in the seminal work by Pierazzi et al. [21] and extended by Apruzzese et al. [12, 22], an attacker operating in the problem space can only manipulate raw packets. Any valid adversarial feature perturbation Δx must correspond to a physical packet transformation $P \to P^*$ that satisfies strict physical conservation laws:

- **Protocol Immutability:** Attributes such as the Destination Port and Transport Protocol cannot be modified by the attacker if they seek to exploit a specific service (e.g., targeting an Apache web server on TCP port 80).
- **Directional Monotonicity:** In physical networks, an attacker can inject additional packets (dummy keep-alives) or insert padding bytes

into application payloads. These actions can only *increase* forward packet counts, forward byte totals, and total flow duration. An attacker cannot subtract bytes below the size of their exploit shellcode, nor can they transmit negative packets.

- **Non-Linear Conservation Laws:** Network flow features are tightly coupled through mathematical relationships. For instance:

$$\text{Flow Bytes/s} = \frac{\text{Total Fwd Bytes} + \text{Total Bwd Bytes}}{\text{Flow Duration} \times 10^{-6}} \tag{3}$$

$$\text{Flow Packets/s} = \frac{\text{Total Fwd Packets} + \text{Total Bwd Packets}}{\text{Flow Duration} \times 10^{-6}} \tag{4}$$

$$\text{Mean Fwd Packet Length} = \frac{\text{Total Fwd Bytes}}{\text{Total Fwd Packets}} \tag{5}$$

If an adversarial algorithm perturbs Total Forward Bytes without simultaneously recalculating Flow Bytes/s and Mean Forward Packet Length, it generates a mathematically inconsistent "phantom flow". When evaluated by a real-world middlebox or stateful intrusion detector, these physically impossible flows are immediately discarded or flagged by rule-based protocol sanity checks.

1.3. The Failure of Existing Paradigms

Faced with these challenges, existing research in adversarial NIDS has remained trapped between two deeply flawed paradigms:

1. **Unconstrained Feature-Space Optimization:** A large body of literature applies optimization algorithms directly in $\mathbb{R}^d$, including exact Mixed-Integer Linear Programming (MILP) solvers [23], Zeroth-Order Optimization (ZOO) [24], and boundary-following search methods [25]. While exact MILP solvers operate in unconstrained feature space (yielding 0% problem-space feasibility by design) and heuristic search algorithms achieve mathematical evasion on paper, they completely ignore protocol coupling. Their generated perturbations routinely violate basic physical conservation laws and monotonicity bounds, resulting in severe Physical Feasibility Rate (PFR) drops (measured at 17.8%–25.0% for benchmarked black-box attacks like ZOO and

HopSkipJump, with unconstrained feature optimization yielding 0.0% Real ASR) when tested against flow conservation checkers [12, 21].

2. **Opaque Packet-Space Heuristics:** At the other extreme, packet-level mutation frameworks such as Traffic Manipulator [26] operate directly on packet captures (PCAPs) using Particle Swarm Optimization (PSO), while constraint-aware tabular search methods (such as CAA and MOEVA [27, 28]) incorporate heuristic box constraints. While raw packet mutators guarantee routability, they treat the NIDS classifier as an opaque black box, requiring hundreds to thousands of queries ($1,240$–$1,850$ queries for evolutionary and swarm optimizers, and 680 queries for HopSkipJump in our benchmark, taking up to several seconds per flow and minutes per batch) and failing to exploit the internal polyhedral partition geometry of tree ensembles.

1.4. Our Proposed Paradigm: Flow-Field Evasion (FFE)

To bridge this fundamental divide between mathematical optimization and physical network reality, this article introduces **Flow-Field Evasion (FFE)**.

Rather than perturbing tabular features arbitrarily or mutating packets blindly, FFE establishes a rigorous differential-geometric foundation for network flow evasion:

1. We model the space of routable network flows through an ambient continuous relaxation $\widetilde{\mathcal{M}}_{\text{flow}} \subset \mathbb{R}^d$ constrained by network conservation laws, coupled with a discrete lattice projection $\mathcal{P}_{\mathcal{M}}$ onto valid packet-derived flow records $\mathcal{M}_{\text{flow}}$.

2. We derive the closed-form **Infinitesimal Generators** (tangent vector fields) for four elementary physical network actions: Temporal Dilation ($\vec{v}_{\text{time}}$), Payload Padding ($\vec{v}_{\text{pad}}$), Chaff Packet Injection ($\vec{v}_{\text{chaff}}$), and TCP Window Morphing ($\vec{v}_{\text{win}}$).

3. These generators define the **Physical Lie Cone** $\mathcal{K}_{\text{net}}(x) \subset T_x\widetilde{\mathcal{M}}_{\text{flow}}$. We prove that integrating non-negative combinations of these generators along $\mathcal{K}_{\text{net}}(x)$ preserves 100% of physical conservation laws and protocol invariants by construction.

4. By evaluating the directional Lie derivative $\mathcal{L}_{\vec{v}_k}F(x)$ of the tree ensemble's decision score along these physical generators, FFE steers attack

flows along the optimal physical Lie-flow trajectory into the benign classification region with minimal query complexity.

1.5. Summary of Key Contributions

The primary contributions of this work are summarized as follows:

- **Physical Lie Flow Algebra:** We formalize the non-linear manifold of valid network flows $\mathcal{M}_{\text{flow}}$ and derive the exact closed-form infinitesimal generators for four elementary packet operations: Temporal Dilation, Payload Padding, Chaff Packet Injection, and TCP Window Morphing.
- **Universal Invariant Preservation Proof:** We prove mathematically that any trajectory generated by integrating non-negative linear combinations of these generators along the positive Lie Cone $\mathcal{K}_{\text{net}}(x)$ preserves 100% of physical flow invariants, packet size bounds, and causal conservation laws by construction.
- **The Flow-Field Evasion (FFE) Algorithm:** We develop a directional Lie derivative steering algorithm that computes $\mathcal{L}_{\vec{v}_k} F(x)$ across the non-differentiable step boundaries of tree ensembles, guiding flows across the nearest polyhedral facets into the benign decision space with minimal operational transmission cost.
- **Multi-Dataset GPU Benchmark:** We implement and benchmark FFE on an NVIDIA RTX 5000 Ada GPU across three diverse benchmark corpora calibrated to empirical marginal distributions of enterprise (CSE-CIC-IDS2018), contemporary hybrid (UNSW-NB15), and cloud OpenStack (CIDDS-001/CIDDS-002) environments—against CUDA-accelerated XGBoost and CPU-multithreaded LightGBM models. FFE consistently achieves 100.0% Attack Success Rate (ASR) and strictly 100.0% Physical Feasibility Rate (PFR) in an average of only $6.8-17.8$ queries (including tree-split snapping evaluations) and under 70 ms latency per flow.
- **Defensive Countermeasures:** We formulate two hardening countermeasures: Malleability-Penalized Tree Splitting (MPS) and Lie-Guided Adversarial Training (LGAT), demonstrating how defenders can mitigate flow-field steering down to an 11.0% evasion rate (from 100% under undefended models to 32.0% under MPS, 18.0% under LGAT, and 11.0% under joint defense).

1.6. Organization of the Paper

The remainder of this paper is structured into five cohesive sections as follows:

- **Section 2: Related Work** surveys the landscape of learning-based NIDS, decision tree verification, problem-space constraints, and constraint-aware evasion benchmarks.
- **Section 3: Problem Formulation & Problem-Space Semantics** formalizes the network flow feature schema, the physical flow manifold $\mathcal{M}_{\text{flow}}$, physical invariance constraints, and the adversary's threat model.
- **Section 4: The Flow-Field Evasion (FFE) Framework** establishes the differential geometry of network flows, derives the infinitesimal Lie generators, establishes the Lie Cone of admissible actions, proves the Invariant Preservation Theorem, and details the algorithmic realization.
- **Section 5: Empirical Benchmark, Ablation Studies & Defenses** presents the GPU benchmark across CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-001 against CUDA XGBoost and LightGBM models across five attack families (with CatBoost and Random Forest evaluated in transferability ablations), benchmarking FFE against five baselines, followed by ablations and defenses (MPS and LGAT).
- **Section 6: Discussion, Real-World Deployment, and Conclusion** analyzes inline packet proxy translation, checksum synchronization, middlebox resilience, operational boundaries, ethical considerations, and concludes the paper.

2. Related Work

The intersection of adversarial machine learning (AML), network security, and decision tree verification has undergone intensive development over the past decade. To contextualize the theoretical and empirical contributions of Flow-Field Evasion (FFE), this section analyzes existing paradigms, tracing their evolution from early unconstrained attacks through problem-space formalisms, exact formal verification solvers, and modern constraint-aware benchmarks.

2.1. Foundations: Machine Learning for Network Telemetry

The application of statistical machine learning to network intrusion detection originated with the realization that signature-based detection engines (such as Snort and Suricata) cannot detect novel, zero-day exploits or polymorphism [10]. Early systems explored unsupervised anomaly detection and shallow classifiers (such as Support Vector Machines, Naive Bayes, and shallow Multi-Layer Perceptrons) trained on legacy corpora like KDD Cup 1999 and NSL-KDD. However, these early models suffered from high false alarm rates, poor scalability, and severe feature representation artifacts [10, 29].

The introduction of modern high-throughput flow extractors (such as CICFlowMeter [1], Zeek, and YAF) coupled with the publication of realistic benchmark datasets—most prominently CSE-CIC-IDS2018 [1], UNSW-NB15 [5], and CIDDS-001/CIDDS-002 [7, 8]—shifted the community toward Gradient Boosted Decision Tree (GBDT) ensembles. Comprehensive empirical studies on ML-based IDS [9, 30, 12] established that tree ensembles—specifically XGBoost [2], LightGBM [3], and CatBoost [4]—represent the de facto gold standard for tabular network telemetry. Their popularity stems from several distinct structural advantages:

1. **Handling Mixed Tabular Types:** Flow records naturally blend continuous metrics (inter-arrival duration, byte rates) with discrete counts (packet totals, header lengths) and categorical attributes (TCP flags, protocols). Decision trees partition continuous and discrete variables natively without requiring complex embedding layers or continuous scaling.

2. **Invariance to Monotonic Transformations:** Unlike deep neural networks, whose linear combinations are sensitive to feature scales and outliers, decision tree split points depend solely on the rank ordering of feature values.

3. **Sub-Millisecond Inference Latency:** A trained tree ensemble can evaluate incoming flow records using simple nested conditional statements (branch comparisons), enabling wire-speed packet classification on standard commodity CPU and GPU hardware.

2.2. Phase I: Naive Feature-Space Transfer

Following the discovery of adversarial examples in deep neural networks by Szegedy et al. [31], Goodfellow et al. [18], and Papernot et al. [32], a

wave of initial studies investigated the vulnerability of machine learning-based NIDS to evasion attacks.

Early works directly applied continuous gradient-based adversarial methods to tabular feature vectors:

- **Fast Gradient Sign Method (FGSM):**

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}\left(\nabla_x \mathcal{L}(f(x), y)\right) \tag{6}$$

- **Projected Gradient Descent (PGD):**

$$x^{(t+1)} = \Pi_{x+\mathcal{S}}\left(x^{(t)} + \alpha \cdot \text{sign}\left(\nabla_x \mathcal{L}(f(x^{(t)}), y)\right)\right) \tag{7}$$

- **Carlini-Wagner (C&W) L_2 Optimization [20]:**

$$\min_{\delta} \|\delta\|_2^2 + c \cdot \max\left(\max_{i \neq t} Z(x+\delta)_i - Z(x+\delta)_t, -\kappa\right) \tag{8}$$

These early studies reported extraordinary attack success rates, frequently exceeding 95%, leading to widespread claims that ML-based NIDS were fundamentally brittle.

However, as subsequent security analyses revealed, these evaluations were fundamentally flawed. By optimizing perturbations $\delta \in \mathbb{R}^d$ inside continuous Euclidean space, the algorithms produced feature vectors that had no physical meaning:

- They assigned fractional values to discrete count metrics (e.g., 23.47 forward packets).
- They perturbed immutable attributes (e.g., changing Destination Port from 80 to 22.3).
- They produced negative inter-arrival times and negative durations.
- They violated physical conservation laws: forward payload byte counts were reduced below the minimum 20-byte TCP header size, or packet counts were increased while byte totals remained zero.

2.3. Phase II: The Problem-Space Paradigm Shift

In their landmark 2020 paper, *"Intriguing Properties of Adversarial ML Attacks in the Problem Space"*, Pierazzi et al. [21] and Verkerken et al. [33] [21] formalized the mathematical framework required to evaluate adversarial attacks in security domains. They demonstrated that an adversarial perturbation is an operational illusion unless it is mapped from the **Problem Space** $\mathcal{Z}$ to the **Feature Space** $\mathcal{X}$.

2.3.1. The Pierazzi Framework

Let $z \in \mathcal{Z}$ denote a real physical object (e.g., an executable binary, a PDF document, or a sequence of network packets). The feature extraction pipeline is a deterministic mapping:

$$\phi : \mathcal{Z} \to \mathcal{X} \tag{9}$$

An adversary cannot manipulate feature vectors directly; they can only apply physical transformations $T \in \mathcal{T}$ in the problem space:

$$z^* = T(z) \tag{10}$$

A problem-space adversarial attack is formally defined as finding a transformation T that satisfies four joint criteria:

1. **Available Transformations:**

   $$T \in \mathcal{T} \tag{11}$$

   representing the real-world manipulation primitives available to the attacker (e.g., packet injection, payload padding, timing dilation).

2. **Classifier Evasion:**

   $$f(\phi(T(z))) \neq y_{\text{true}} \tag{12}$$

3. **Semantic Preservation:**

   $$\mathcal{S}(T(z)) = \mathcal{S}(z) \tag{13}$$

   meaning the malicious utility, exploit payload, or botnet command-and-control functionality must execute successfully on the target system.

4. **Plausibility and Robustness to Preprocessing:** $T(z)$ must adhere to valid protocol syntax and format specifications (e.g., valid IP/TCP checksums, routable packet headers) so that intermediate networking devices (routers, firewalls, middleboxes) do not drop or normalize the transmission before it reaches the NIDS sensor.

2.3.2. Empirical Audits by Apruzzese et al.

Following the formalization of Pierazzi et al. [21] and Verkerken et al. [33], Apruzzese et al. [12, 22, 13, 14] critically examined the adversarial NIDS literature, establishing that the vast majority of published feature-space attacks rely on unfeasible threat models:

- Standard feature-space perturbations routinely generate feature vectors that cannot be realized via valid network packets under RFC protocol specifications.
- Real-world network telemetry is extracted by stateful network parsers; unconstrained coordinate displacements that disregard flow conservation laws fail to survive physical feature extraction.
- Consequently, when evaluated against true packet parsers and physical middlebox filters, the reported attack success rates of conventional feature-space AML attacks degrade catastrophically.

2.3.3. Tabular Constrained Benchmarks: CAA and TabularBench

To address these limitations within tabular domains, Simonetto et al. [27] introduced the **Constrained Adaptive Attack (CAA)**, which combined two complementary engines:

1. **CAPGD (Constrained Adaptive PGD):** A parameter-free gradient attack that projects continuous perturbations onto user-defined hyper-rectangular bounding boxes $[x_i^{\min}, x_i^{\max}]$ and enforces feature immutability.
2. **MOEVA (Multi-Objective Evolutionary Adversarial Attack [28]):** A genetic algorithm based on R-NSGA-III that optimizes a multi-objective fitness function penalizing both classifier detection confidence and constraint violation penalties.

Subsequently, Simonetto et al. [34] formalized these methods into **TabularBench**, establishing a standardized benchmark across deep learning and tree ensembles. While CAA represented significant progress over unconstrained PGD, it exhibited practical limitations in network telemetry:

- While MOEVA's domain-specific constraint language allows formulating non-linear penalties, genetic optimization on coupled rational conservation laws (Rate $= \frac{\text{Bytes}}{\text{Duration}}$) requires dense stochastic population

exploration without directional derivative guidance, resulting in frequent intermediate invariant violations and prohibitive query complexity ($> 1,000$ queries per sample).

- CAPGD relies on axis-aligned hyper-rectangular clipping $[x_i^{\min}, x_i^{\max}]$, which cannot preserve non-linear hyperbolic surfaces ($\Psi(x) = \mathbf{0}$) where coordinate changes must be tightly coupled at every step.

2.4. Phase III: Tree Ensemble Solvers and Formal Verification

Because Gradient Boosted Decision Trees are piecewise-constant, researchers sought exact mathematical representations of tree decision boundaries.

2.4.1. Kantchelian's Exact MILP Evasion

In their foundational paper, Kantchelian et al. [23] formulated adversarial evasion against tree ensembles as a Mixed-Integer Linear Program (MILP).

Given an ensemble of T trees, where each tree t has leaves L_t, a binary indicator variable $z_{t,l} \in \{0, 1\}$ is defined for each leaf node $l \in L_t$. Each leaf corresponds to a hyper-rectangular cell defined by split predicates:

$$x_j \leq \theta_j \quad \text{or} \quad x_j > \theta_j \tag{14}$$

The exact minimal L_1 or L_0 perturbation is found by solving:

$$\min_{x^*, z} \quad \|x^* - x_0\|_p \tag{15}$$

$$\text{subject to} \quad \sum_{t=1}^{T} \sum_{l \in L_t} w_{t,l} z_{t,l} < \tau \tag{16}$$

$$\sum_{l \in L_t} z_{t,l} = 1, \quad \forall t \in \{1, \ldots, T\} \tag{17}$$

$$z_{t,l} = 1 \implies x^* \in \text{Box}(l) \tag{18}$$

While mathematically exact, Kantchelian's MILP formulation suffers from severe practical limitations:

1. **Combinatorial Complexity:** The number of binary variables scales with the total number of leaves across all trees $\sum_{t=1}^{T} |L_t| \approx T \cdot 2^D$. For modern production ensembles with $T = 100$ trees and depth $D = 8$, the solver must reason over $25,600$ integer variables. The problem is NP-hard with worst-case complexity $2^{\mathcal{O}(D \cdot T)}$, and standard solvers (such as Gurobi or CPLEX) frequently timeout or exhaust system memory.

2. **Linearity Restriction:** MILP solvers natively accept only linear constraints. When non-linear network flow invariants are introduced (such as non-linear rate equations relating bytes and duration), the formulation becomes a Mixed-Integer Non-Linear Program (MINLP), which is generally intractable.

2.4.2. SAT, SMT, and Continuous Leaf Tree Attacks

Since the early 2020s, researchers adapted Boolean Satisfiability (SAT) and Satisfiability Modulo Theories (SMT) to formally verify and explain GBDT ensembles. Ignatiev et al. [35] and Izza et al. [36] encode tree ensemble splits into propositional Horn and MaxSAT clauses to synthesize minimal, non-redundant formal explanations and analyze robustness properties for decision trees and ensembles. In parallel, Zhang et al. [37] introduced LT-Attack, an efficient white-box attack against tree ensembles based on continuous leaf approximations, while Chen et al. [38], Andriushchenko & Hein [39], and Calzavara et al. [40] proposed robust tree training objectives.

However, formal SMT verification encounters the same fundamental barrier: reasoning over large ensembles requires massive combinatorial state exploration, and non-linear rational constraints (such as rate equations $x_8 = B_{\text{tot}}/T_{\text{sec}}$) exceed the practical scalability of non-linear real arithmetic (NRA) solvers. Meanwhile, continuous leaf approximations (LT-Attack) operate in unconstrained Euclidean space, ignoring coupled network conservation laws and protocol feasibility.

2.5. Phase IV: Black-Box Heuristic Traffic Mutation

To circumvent mathematical feature-space abstractions entirely, security practitioners developed packet-level mutation engines:

- **Traffic Manipulator (Han et al. [26]):** Uses Particle Swarm Optimization (PSO) to mutate raw PCAP files. It perturbs packet transmission intervals, injects dummy payload bytes, and introduces chaff packets.
- **Adversarial Feature Generation (Lin et al. [41]):** Employs Wasserstein Generative Adversarial Networks (IDSGAN) to synthesize adversarial tabular flow records approximating benign distributions in feature space.

- **Deep Traffic Detection under Packet Morphing:** Evaluated how deep and tree-based traffic anomaly detectors are vulnerable to timing and volumetric morphing under realistic problem-space constraints (e.g., Apruzzese et al. [12, 22, 13, 14]).

While packet mutation frameworks guarantee physical routability by operating directly on packets, their optimization engines treat the target classifier as an opaque black box. Because Genetic Algorithms and PSO randomly mutate parameters without knowing which tree splits are active, they require hundreds to thousands of queries ($1,240$–$1,850$ queries for evolutionary and swarm optimizers, and 680 queries for HopSkipJump in our benchmark, taking up to several seconds per flow and minutes per batch).

2.6. Taxonomic Comparison of Evasion and Verification Paradigms

Table 1 summarizes representative evasion, verification, and defense methods developed across the literature, comparing target architectures, optimization mechanisms, query complexities, and problem-space feasibility.

Table 1: Taxonomic Comparison of Representative Adversarial Evasion and Verification Paradigms for Tabular and Network Telemetry (Empirical PFR percentages denote evaluation results on calibrated network flow telemetry; see Section 5.2)

Paper / Method	Target Model	Optimization Engine	Query Complexity	Physical Feasibility	Year
Kantchelian et al. [23]	GBDT / Random Forest	Mixed-Integer Linear Program (MILP)	Zero (White-Box)	Unconstrained (Feature Space)	2016
Carlini & Wagner [20]	Deep Neural Networks	Lagrangian L_2/L_∞ Optimization	$O(N_{\text{iter}})$	Unconstrained (Feature Space)	2017
Chen et al. (ZOO) [24]	Black-Box Classifiers	Zeroth-Order Coordinate Descent	$O(d \cdot N_{\text{iter}})$	25.0% (Empirical Benchmark)	2017
Pierazzi et al. [21] and Verkerken et al. [33] [21]	Android / Problem Space	Problem-Space Formalization	Analytic	Formally Defined	2020
Hashemi & Keller [42]	Neural NIDS	Adversarial Retraining / RePO	$O(N_{\text{train}})$	Unconstrained (Feature Space)	2020
Chen et al. (HSJ) [25]	Black-Box Decision	Boundary Random Walk	$O(10^3)$	17.8% (Empirical Benchmark)	2020
Zhang et al. (LT-Attack) [37]	Tree Ensembles	Continuous Leaf Approximation	$O(N_{\text{iter}})$	Unconstrained (Feature Space)	2020
Ghamizi et al. (MOEVA) [28]	Constrained Tabular	Multi-Objective Genetic Search	$O(10^4)$	Partial (Heuristic Penalty)	2020
Han et al. (Traffic Manipulator) [26]	Black-Box NIDS	Particle Swarm Optimization (PCAP)	$O(10^3 - 10^4)$	100% (Packet Mutation)	2021
Lin et al. (IDSGAN) [41]	Black-Box NIDS	Generative Adversarial Network	$O(N_{\text{gen}})$	Approximated (Feature Space)	2022
Apruzzese et al. [12, 22]	Learning-Based NIDS	Problem-Space Adversary Audits	Empirical Audit	Real Testbed / Packets	2022/2023
Ignatiev et al. [35]	GBDT Ensembles	Boolean MaxSAT / SMT Verification	Zero (White-Box)	Formal Explanation	2022
Izza et al. [36]	Decision Trees	MaxSAT Explanation Redundancy	Zero (White-Box)	Formal Explanation	2022
Simonetto et al. (CAA) [27]	Tabular Models	CAPGD + Heuristic Snapping	$O(10^3)$	Partial (61.5% Empirical)	2024
Simonetto et al. (TabularBench) [34]	Tabular Benchmarks	Standardized Robustness Framework	$O(10^3)$	Partial (Bounded Domain)	2024
Flow-Field Evasion (Ours)	**XGBoost / LightGBM**	**Lie Vector Field Steering**	**12.9 (Low)**	**100.0% (Guaranteed)**	**2026**

2.7. The Critical Unresolved Gap

As evidenced by Table 1, the existing literature leaves a profound gap in the scientific landscape:

1. Methods that exploit mathematical structure (MILP, SMT, unconstrained AML) operate in unconstrained feature space, generating non-routable perturbations that cause Real ASR to collapse (down to 0.0% to 8.0% Real ASR across unconstrained baselines).

2. Methods that preserve physical feasibility (Traffic Manipulator, live packet mutators) treat the classifier as an opaque black box, resulting in elevated query complexities ($680 - 1,850$ queries in our benchmark) and multi-second latencies.

Flow-Field Evasion (FFE) resolves this dilemma. By deriving the exact infinitesimal tangent vector fields directly from elementary physical packet operations, FFE conducts gradient-like directed search on tree decision scores while remaining mathematically confined to the physical flow manifold $\mathcal{M}_{\text{flow}}$ at every step.

3. Problem Formulation & Problem-Space Semantics

This section formalizes the mathematical relationship between the physical problem space of network packet transmissions and the feature space of tabular flow telemetry. We define the properties of the feature aggregation mapping, establish the formal structure of the network flow manifold $\mathcal{M}_{\text{flow}}$, and specify the adversary's formal threat model.

3.1. Packet Streams and Network Protocols

Let $\Omega = \{0,1\}^*$ denote the space of all binary payloads. A single network packet p is a structured tuple:

$$p = (\text{hdr}_{\text{link}}, \text{hdr}_{\text{net}}, \text{hdr}_{\text{trans}}, \text{payload}) \in \Omega \tag{19}$$

complying with standard layered protocol architectures (Ethernet at the link layer, IPv4/IPv6 at the network layer, and TCP/UDP at the transport layer).

A packet stream (or transmission sequence) P is an ordered sequence of m timestamped packets:

$$P = \big((p_1, t_1), (p_2, t_2), \ldots, (p_m, t_m)\big) \tag{20}$$

where $t_i \in \mathbb{R}_{\geq 0}$ represents the physical transmission timestamp at the network interface card (NIC). The sequence satisfies strict temporal causality:

$$0 \leq t_1 \leq t_2 \leq \cdots \leq t_m \tag{21}$$

3.1.1. The Space of Valid Packet Streams $\mathcal{P}_{valid}$

Not all arbitrary bit sequences are admissible in real-world networking. We denote by $\mathcal{P}_{\text{valid}} \subset (\Omega \times \mathbb{R}_{\geq 0})^*$ the set of packet streams that satisfy three mandatory physical constraints:

1. **Syntactic RFC Compliance:** Every packet p_i must have valid protocol header fields, matching length declarations, and correct Internet Checksums (RFC 791 [43] IPv4 header checksum, RFC 1071 [44] checksum calculation, and RFC 9293 [45] TCP / RFC 768 [46] UDP checksums; for UDP over IPv4, the transport checksum is either computed or legitimately zero):

$$\text{ValidChecksum}_{\text{IP}}(p_i) = \text{True}, \quad \text{ValidChecksum}_{\text{Transport}}(p_i) = \text{True} \tag{22}$$

   Packets with corrupted checksums are discarded upon ingress by standard network stacks and intermediate middleboxes before reaching an intrusion detection sensor.

2. **Transport State Machine Consistency:** For TCP traffic, packets must respect the TCP finite state machine (RFC 9293 [45]): for consecutive in-order forward segments without retransmission, sequence numbers (*seq*) and acknowledgment numbers (*ack*) advance according to transmitted payload bytes and handshake flags:

$$seq_{i+1} = seq_i + \text{Len}(\text{payload}_i) + \mathbb{I}_{\text{SYN/FIN}} \tag{23}$$

3. **Maximum Transmission Unit (MTU) Bounds:** The total length of any single physical IP packet $\text{Len}_{\text{IP}}(p_i)$ (including IP header, transport header, and payload) cannot exceed the network interface IP MTU (1,500 bytes for standard Ethernet per RFC 894 [47] and RFC 791 [43], corresponding to link-layer frame size $\leq$ 1514 bytes):

$$\forall i \in \{1, \ldots, m\}, \quad \text{Len}_{\text{IP}}(p_i) \leq \text{MTU}_{\text{IP}} = 1500 \text{ bytes} \tag{24}$$

3.2. Flow-Based Telemetry Feature Mapping Φ

A network flow is defined as a bidirectional sequence of packets sharing a common 5-tuple socket identifier:

$$\text{FlowID} = (\text{IP}_{\text{src}}, \text{IP}_{\text{dst}}, \text{Port}_{\text{src}}, \text{Port}_{\text{dst}}, \text{Proto}) \tag{25}$$

Packets are separated into two directional sub-streams:

- **Forward Direction (Fwd):** Packets originating from the client initiator toward the destination server.
- **Backward Direction (Bwd):** Response packets originating from the destination server toward the client.

A flow exporter (e.g., CICFlowMeter [1]) computes a deterministic, many-to-one aggregation mapping:

$$\Phi : \mathcal{P}_{\text{valid}} \to \mathbb{R}^d \tag{26}$$

The feature vector $x = \Phi(P) \in \mathbb{R}^d$ summarizes the statistical, volumetric, and temporal dynamics of the transmission.

3.3. The 12-Dimensional Flow Feature Schema

To formulate our analysis concretely, we focus on a representative, standardized 12-dimensional feature schema capturing the core metrics analyzed by state-of-the-art NIDS models on CSE-CIC-IDS2018 [1]. The feature vector $x \in \mathbb{R}^{12}$ is structured as follows:

x_0: **Destination Port (dst_port):** Service port on the target machine (e.g., 80 for HTTP, 443 for HTTPS, 22 for SSH).

x_1: **Protocol (protocol):** Transport layer protocol identifier (TCP = 6, UDP = 17).

x_2: **Total Forward Packets (N_{fwd}):** The total number of packets transmitted in the forward client-to-server direction:

$$x_2 = |\{p_i \in P \mid \text{Dir}(p_i) = \text{Fwd}\}| \tag{27}$$

x_3: **Total Backward Packets (N_{bwd}):** The total number of response packets received from the server.

x_4: **Total Length of Forward Packets (B_{fwd}):** Total cumulative wire bytes (IP packet lengths including IP/transport headers and payload) transmitted in the forward direction:

$$x_4 = \sum_{p_i \in P_{\text{fwd}}} \text{Len}(p_i) \tag{28}$$

We note that while some CICFlowMeter releases record application payload bytes exclusively for this attribute, x_4 in our schema adheres to IPFIX/NetFlow wire-level standards by measuring total IP-layer forward bytes (inclusive of IP and transport headers), providing an algebraically consistent physical conservation foundation with header length (x_{11}).

x_5: **Total Length of Backward Packets ($B_{\mathbf{bwd}}$):** Total bytes received in the backward direction.

x_6: **Flow Duration (T):** Total elapsed time of the flow in microseconds:

$$x_6 = (t_m - t_1) \times 10^6 \quad (\mu\text{s}) \tag{29}$$

x_7: **Initial Forward Window Bytes ($W_{\mathbf{fwd}}$):** TCP receive window size advertised in the initial client forward SYN packet ($0 \leq x_7 \leq 65535$ for TCP flows with $x_1 = 6$; for UDP flows with $x_1 = 17$, $x_7 \equiv 0$ as UDP is connectionless).

x_8: **Flow Bytes/s (R_B):** The overall transmission rate in bytes per second:

$$x_8 = \frac{x_4 + x_5}{x_6 \times 10^{-6}} = \frac{10^6 \cdot (x_4 + x_5)}{x_6} \tag{30}$$

x_9: **Flow Packets/s (R_P):** The overall packet frequency in packets per second:

$$x_9 = \frac{x_2 + x_3}{x_6 \times 10^{-6}} = \frac{10^6 \cdot (x_2 + x_3)}{x_6} \tag{31}$$

x_{10}: **Forward Packet Length Mean ($\bar{S}_{\mathbf{fwd}}$):** Average size of forward packets in bytes:

$$x_{10} = \frac{x_4}{\max(1, x_2)} \tag{32}$$

x_{11}: **Forward Header Length ($H_{\mathbf{fwd}}$):** Cumulative size of transport headers in the forward direction. Because minimum transport header sizes are 20 bytes for TCP ($x_1 = 6$) and 8 bytes for UDP ($x_1 = 17$):

$$x_{11} \geq x_2 \times (20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}) \text{ bytes} \tag{33}$$

Similarly, forward packet bytes x_4 include the 20-byte IPv4 header plus transport header and payload, bounded by the physical Ethernet

Maximum Transmission Unit (IP MTU ≤ 1500 bytes, equating to link-layer frame size ≤ 1514 bytes):

$$x_2 \times (40 \cdot \mathbb{I}_{x_1=6} + 28 \cdot \mathbb{I}_{x_1=17}) \leq x_4 \leq 1500 x_2 \text{ bytes} \tag{34}$$

3.4. Formal Categorization of Feature Invariants

We categorize the feature indices $\mathcal{D} = \{0, 1, \ldots, 11\}$ into five operational constraint classes based on physical network semantics:

3.4.1. Immutable Features $\mathcal{I}_{imm} = \{0, 1, 3, 5\}$

These attributes are strictly fixed. Destination port ($x_0 \in \{1, \ldots, 65535\}$) and protocol ($x_1 \in \{6, 17\}$) cannot be changed without redirecting the attack to an unintended service. Backward packets (x_3) and backward bytes (x_5) are generated by the target server; an external client cannot directly control server responses. Thus:

$$\forall i \in \mathcal{I}_{\text{imm}}, \quad \Delta x_i = 0 \tag{35}$$

3.4.2. Monotonically Non-Decreasing Features $\mathcal{I}^{+}_{mono} = \{2, 4, 6, 11\}$

An attacker can inject dummy packets ($\Delta N_{\text{fwd}} \geq 0$, incrementing packet count x_2 and header length x_{11} by $H_0 \Delta N_{\text{fwd}}$), pad payloads with null bytes ($\Delta B_{\text{fwd}} \geq 0$, incrementing forward byte volume x_4), or delay transmissions ($\Delta T \geq 0$, dilating duration x_6). However, an attacker cannot subtract payload bytes required to deliver their malicious shellcode, nor can they transmit negative packets:

$$\forall j \in \mathcal{I}^{+}_{\text{mono}}, \quad \Delta x_j \geq 0 \tag{36}$$

3.4.3. Bounded Continuous Features $\mathcal{I}_{bound} = \{7\}$

The TCP receive window size x_7 can be modulated within operating system protocol limits for TCP flows ($0 \leq x_7 + \Delta x_7 \leq 65535$ if $x_1 = 6$; for UDP flows with $x_1 = 17$, $x_7 \equiv 0$ is fixed):

$$0 \leq x_7 + \Delta x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6} \tag{37}$$

3.4.4. Equality-Coupled Derived Features $\mathcal{I}_{dep} = \{8, 9, 10\}$

Features x_8, x_9, x_{10} are deterministic, non-linear algebraic functions of base metrics:

$$x_8 = g_8(x_4, x_5, x_6) = \frac{10^6 \cdot (x_4 + x_5)}{x_6} \tag{38}$$

$$x_9 = g_9(x_2, x_3, x_6) = \frac{10^6 \cdot (x_2 + x_3)}{x_6} \tag{39}$$

$$x_{10} = g_{10}(x_4, x_2) = \frac{x_4}{\max(1, x_2)} \tag{40}$$

3.4.5. Structural Inequality Invariants $\mathcal{I}_{ineq} = \{4, 11\}$

Features x_4 and x_{11} satisfy protocol bounding inequalities: $x_{11} \geq x_2 \times (20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17})$, $x_4 \geq x_{11} + 20x_2 \geq x_2 \times (40 \cdot \mathbb{I}_{x_1=6} + 28 \cdot \mathbb{I}_{x_1=17})$, with MTU ceiling $x_4 \leq 1500x_2$. Note that operational degrees of freedom may couple multiple categories: for instance, forward byte volume x_4 is both monotonically non-decreasing ($\Delta x_4 \geq 0$) and bounded by structural MTU inequalities ($L_0 x_2 \leq x_4 \leq 1500x_2$), while forward header bytes x_{11} is monotonically non-decreasing ($\Delta x_{11} \geq 0$) under chaff injection. In Algorithm 4, $\mathcal{I}^+_{\text{mono}} = \{2, 4, 6, 11\}$ represents the mutable monotonic features.

3.5. The Flow Manifold $\mathcal{M}_{flow}$ *and Ambient Relaxation* $\widetilde{\mathcal{M}}_{flow}$

The physical packet-level flow space is defined as the discrete image of valid packet streams under the telemetry exporter:

$$\mathcal{M}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \exists P \in \mathcal{P}_{\text{valid}} \text{ s.t. } \Phi(P) = x \right\} \subset \mathbb{Z}^9_{\geq 0} \times \mathbb{R}^3_{\geq 0} \tag{41}$$

where discrete coordinates $(x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_{11})$ satisfy integrality (with packet counts $x_2 \in \mathbb{N}_{\geq 1}, x_3 \in \mathbb{N}_{\geq 0}$, duration in integer microseconds $1 \leq x_6 \leq 1.1 \times 10^8\,\mu$s bounded below the 120-second exporter timeout), rates $(x_8, x_9, x_{10}) \in \mathbb{R}^3_{\geq 0}$, and packet lengths comply with interface MTU restrictions ($x_4 \leq 1500x_2$). All formal definitions and evaluations focus on IPv4 ($L_{\text{IPv4}} = 20$ bytes) and standard Ethernet MTU per RFC 791 and RFC 894 [43, 47].

To enable continuous differential geometric optimization, we define the *continuous ambient flow manifold* $\widetilde{\mathcal{M}}_{\text{flow}} \subset \mathbb{R}^{12}$ by relaxing packet counts and duration to positive reals ($x_2 \geq 1, x_6 > 0$), with transport header limits

$H_{\min}(x_1) = 20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$ and $H_{\max}(x_1) = 60 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$:

$$\widetilde{\mathcal{M}}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \begin{array}{l} x_0 \in [1, 65535], \quad x_1 \in \{6, 17\}, \quad x_2 \geq 1, \quad x_3, x_5 \geq 0, \\ H_{\min}(x_1)x_2 \leq x_{11} \leq H_{\max}(x_1)x_2, \\ x_{11} + 20x_2 \leq x_4 \leq 1500x_2, \quad 0 < x_6 \leq 1.1 \times 10^8, \\ 0 \leq x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6}, \quad x_{10} = x_4/x_2, \\ x_8 = 10^6(x_4 + x_5)/x_6, \quad x_9 = 10^6(x_2 + x_3)/x_6 \end{array} \right\} \tag{42}$$

Continuous vector field flows are formulated on $\widetilde{\mathcal{M}}_{\text{flow}}$, while discrete execution snaps continuous trajectories back to $\mathcal{M}_{\text{flow}}$ via the closed-form projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 4.2).

3.6. Formal Threat Model

We formalize the attacker's operational game under standard adversarial machine learning criteria:

- **Adversary's Goal:** Given a malicious flow $x_{\text{orig}} = \Phi(P) \in \mathcal{M}_{\text{flow}}$ classified as an attack ($P(\text{Malicious} \mid x_{\text{orig}}) \geq 0.5 \iff F(x_{\text{orig}}) \geq 0$), synthesize a modified flow $x^* = \Phi(P^*) \in \mathcal{M}_{\text{flow}}$ such that:

$$P(\text{Malicious} \mid x^*) < 0.5 \iff F(x^*) < 0 \tag{43}$$

- **Adversary's Knowledge (Two-Tier Model):** We evaluate two operational threat tiers:

  1. *Tier 1 (White-Box Tree Access):* The adversary has full access to the ensemble architecture, leaf weights, and split thresholds Θ_j, enabling exact Tree-Split Boundary Snapping (TSBS). The primary comparative benchmark (Section 5.2) evaluates this tier.
  2. *Tier 2 (Black-Box Transfer / Query Access):* The adversary lacks internal access to target model structures; evasion trajectories are synthesized either via black-box directional exploratory probes or crafted on a local surrogate tree ensemble and transferred zero-knowledge to unseen target architectures. The cross-architecture transferability benchmark (Table 11) evaluates this tier.

- **Functionality Preservation:** The transformed packet trace P^* must preserve the original malicious semantics:

$$\mathcal{S}(P^*) = \mathcal{S}(P) \tag{44}$$

meaning exploit delivery, shellcode execution, authentication cracking, or target service degradation must succeed on the victim target. Under Pierazzi's problem-space formulation, malicious utility preservation $\mathcal{S}(P^*) = \mathcal{S}(P)$ operates across two regimes:

  - *Discrete Payload Exploits (SSH Brute Force, Web attacks, Infiltration):* Payload padding ($\vec{v}_{\text{pad}}$) appends non-executable trailing bytes or protocol-compliant padding (e.g., `SSH_MSG_IGNORE` per RFC 4253 [48]) without modifying exploit byte sequences or credentials, guaranteeing exact functional execution.
  - *Volumetric Attacks (DoS/DDoS, UDP floods):* Temporal dilation ($\vec{v}_{\text{time}}$) spreads packet transmissions over an expanded duration; while this trades peak instantaneous bandwidth flooding for persistent evasion, all packets reach the target service, consuming connection table slots, server memory, and thread pool resources (analogous to low-and-slow DoS attacks like Slowloris), preserving service degradation capability while evading volumetric rate filters.

- **Deployment Architecture (Ingress Perimeter Biflow Inspection):** In alignment with enterprise boundary defense deployments, the NIDS operates as a bidirectional telemetry inspection sensor (e.g., NetFlow v9 / IPFIX biflow collector per RFC 5103 at an enterprise border router or DMZ firewall) that monitors aggregated flow records. While the sensor aggregates both transmission directions into the flow record, the adversary only controls the client-side attack endpoint. Consequently, the adversary actively manipulates forward flow parameters $(x_2, x_4, x_6, x_7, x_{11})$ and the resulting coupled rates (x_8, x_9, x_{10}) via inline proxy mechanics or endpoint socket options, while backward attributes (x_3, x_5) reflect autonomous external server responses and are treated as strictly immutable ($\mathcal{I}_{\text{imm}}$). Forward chaff packets are transmitted as unidirectional forward pure ACKs carrying zero application payload, current sequence number seq = SND.NXT, and valid acknowledgment ack = RCV.NXT. Under RFC 9293 §3.8.1, a TCP endpoint must

never generate an acknowledgment in response to receiving a pure ACK (preventing infinite ACK loops), guaranteeing that forward pure ACKs elicit strictly zero backward packets or bytes ($\Delta x_3 = 0, \Delta x_5 = 0$). Furthermore, to avoid triggering peer fast retransmit or spurious loss recovery under RFC 5681 §3.2 [49] (which requires three duplicate ACKs), forward chaff segments are bounded or interleaved with forward payload segments, window updates, or spaced beyond duplicate ACK thresholds. Window morphing ($\vec{v}_{\text{win}}$) specifically modifies the initial SYN packet header window field (synchronized with RFC 7323 [50] WSCALE) prior to transmission. For protocols involving application-layer encryption (such as SSH), application payload padding (e.g., RFC 4253 §11.2 [48] `SSH_MSG_IGNORE`) is applied at the client application/socket layer prior to cryptographic encapsulation.

- **Minimal Transmission Cost:** The adversary seeks to minimize operational transmission overhead:

$$\min \quad \mathcal{C}(x^*, x_{\text{orig}}) = \sum_{k=1}^{4} c_k s_k \tag{45}$$

where c_k denotes the physical operational unit cost of executing generator k (e.g., transmission latency, bandwidth expansion, socket buffer allocation).

4. The Flow-Field Evasion (FFE) Framework

4.1. Differential Geometry of Network Flows: The Physical Lie Cone

To resolve the conflict between discrete decision trees and continuous physical flow invariants, this section establishes a formal differential-geometric framework for network flow evasion. We formalize the tangent space $T_x\mathcal{M}_{\text{flow}}$, derive the closed-form infinitesimal generators of physical network transformations, analyze their Lie bracket commutators, and prove that integrating along the positive Lie Cone guarantees absolute physical feasibility. Figure 1 illustrates the high-level system architecture and mathematical trajectory of the proposed framework.

4.1.1. The Continuous Relaxation $\widetilde{\mathcal{M}}_{flow}$ and Tangent Space $T_x\widetilde{\mathcal{M}}_{flow}$

While the physical telemetry space $\mathcal{M}_{\text{flow}} = \Phi(\mathcal{P}_{\text{valid}}) \subset \mathbb{Z}_{\geq 0}^9 \times \mathbb{R}_{\geq 0}^3$ is a discrete packet-level lattice embedded in $\mathbb{R}^{12}$ (where packet counts, byte totals,

socket ports, TCP window, header lengths, and integer microsecond duration satisfy $(x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_{11}) \in \mathbb{Z}^9_{\geq 0}$, while rates $(x_8, x_9, x_{10}) \in \mathbb{R}^3_{\geq 0}$), we analyze its differential geometry via the smooth continuous ambient relaxation $\widetilde{\mathcal{M}}_{\text{flow}}$ (Section 3) where $\Psi(x) = \mathbf{0}$, $x_2 \geq 1$, $0 < x_6 \leq 1.1 \times 10^8$, $x_{11} \geq x_2(20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17})$, and $x_{11} + 20x_2 \leq x_4 \leq 1500x_2$. The coupled conservation equations defining $\widetilde{\mathcal{M}}_{\text{flow}}$ can be expressed as the zero level-set of a smooth vector-valued constraint function $\Psi : \mathbb{R}^{12} \to \mathbb{R}^3$:

$$\Psi(x) = \begin{bmatrix} \Psi_1(x) \\ \Psi_2(x) \\ \Psi_3(x) \end{bmatrix} = \begin{bmatrix} x_8 - \dfrac{10^6(x_4 + x_5)}{x_6} \\ x_9 - \dfrac{10^6(x_2 + x_3)}{x_6} \\ x_{10} - \dfrac{x_4}{x_2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{46}$$

By the Regular Value Theorem, at any regular point $x \in \widetilde{\mathcal{M}}_{\text{flow}}$ where the Jacobian matrix $J_\Psi(x) \in \mathbb{R}^{3\times 12}$ has full rank 3, the tangent space $T_x\widetilde{\mathcal{M}}_{\text{flow}}$ is given by the kernel of the Jacobian:

$$T_x\widetilde{\mathcal{M}}_{\text{flow}} = \ker\left(J_\Psi(x)\right) = \left\{\vec{v} \in \mathbb{R}^{12} \;\middle|\; J_\Psi(x) \cdot \vec{v} = 0\right\} \tag{47}$$

Computing the partial derivatives of Ψ:

$$\nabla\Psi_1(x) = \left[0, 0, 0, 0, -\frac{10^6}{x_6}, -\frac{10^6}{x_6}, \frac{10^6(x_4 + x_5)}{x_6^2}, 0, 1, 0, 0, 0\right] \tag{48}$$

$$\nabla\Psi_2(x) = \left[0, 0, -\frac{10^6}{x_6}, -\frac{10^6}{x_6}, 0, 0, \frac{10^6(x_2 + x_3)}{x_6^2}, 0, 0, 1, 0, 0\right] \tag{49}$$

$$\nabla\Psi_3(x) = \left[0, 0, \frac{x_4}{x_2^2}, 0, -\frac{1}{x_2}, 0, 0, 0, 0, 0, 1, 0\right] \tag{50}$$

Any physically realizable velocity vector $\vec{v} = [\dot{x}_0, \ldots, \dot{x}_{11}]^T \in T_x\widetilde{\mathcal{M}}_{\text{flow}}$ must satisfy the linearized conservation laws:

$$\dot{x}_8 = \frac{10^6(\dot{x}_4 + \dot{x}_5)}{x_6} - \frac{10^6(x_4 + x_5)}{x_6^2}\dot{x}_6 = \frac{10^6(\dot{x}_4 + \dot{x}_5)}{x_6} - \frac{x_8}{x_6}\dot{x}_6 \tag{51}$$

$$\dot{x}_9 = \frac{10^6(\dot{x}_2 + \dot{x}_3)}{x_6} - \frac{10^6(x_2 + x_3)}{x_6^2}\dot{x}_6 = \frac{10^6(\dot{x}_2 + \dot{x}_3)}{x_6} - \frac{x_9}{x_6}\dot{x}_6 \tag{52}$$

$$\dot{x}_{10} = \frac{\dot{x}_4}{x_2} - \frac{x_4}{x_2^2}\dot{x}_2 = \frac{\dot{x}_4 - x_{10}\dot{x}_2}{x_2} \tag{53}$$

4.1.2. Closed-Form Derivation of Infinitesimal Generators

An adversary controls four elementary operational degrees of freedom. Each degree of freedom defines a 1-parameter flow $\{\phi_k^s\}_{s\geq 0}$ whose infinitesimal push-forward induces a smooth tangent vector field on $\widetilde{\mathcal{M}}_{\text{flow}}$:

1. Temporal Dilation Field $\vec{v}_{time}(x)$.. The adversary introduces transmission delays ΔT between packets, dilating flow duration without injecting additional bytes or packets ($\dot{x}_2 = \dot{x}_3 = \dot{x}_4 = \dot{x}_5 = 0, \dot{x}_6 = 1$). Substituting into Equations (51)–(53):

$$\dot{x}_8 = -\frac{x_8}{x_6} \tag{54}$$

$$\dot{x}_9 = -\frac{x_9}{x_6} \tag{55}$$

$$\dot{x}_{10} = 0 \tag{56}$$

All other components are identically zero. Hence, the temporal dilation generator is:

$$\vec{v}_{\text{time}}(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ -x_8/x_6 \\ -x_9/x_6 \\ 0 \\ 0 \end{bmatrix} \tag{57}$$

2. Payload Padding Field $\vec{v}_{pad}(x)$.. The adversary appends dummy padding bytes ΔB_{fwd} to forward packets without altering packet counts or flow duration ($\dot{x}_4 = 1, \dot{x}_2 = \dot{x}_3 = \dot{x}_5 = \dot{x}_6 = 0$). Substituting into the tangent

equations:

$$\dot{x}_8 = \frac{10^6}{x_6} \tag{58}$$

$$\dot{x}_9 = 0 \tag{59}$$

$$\dot{x}_{10} = \frac{1}{x_2} \tag{60}$$

Thus, the payload padding generator is:

$$\vec{v}_{\text{pad}}(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 10^6/x_6 \\ 0 \\ 1/x_2 \\ 0 \end{bmatrix} \tag{61}$$

3. Chaff Packet Injection Field $\vec{v}_{chaff}(x)$.. The adversary transmits dummy forward packets ΔN_{fwd}, each carrying standard transport headers with zero application payload (e.g., TCP keep-alives or pure ACKs per RFC 9293 §3.8.4 [45]: for minimal empty TCP segments without options, $L_0 = 40, H_0 = 20$; when RFC 7323 [50] TCP timestamps are negotiated, headers include the 12-byte TSOPT, setting $L_0 = 52, H_0 = 32$; for UDP per RFC 768 [46], 20 bytes IP header + 8 bytes UDP header, setting $L_0 = 28, H_0 = 8$; with $\dot{x}_2 = 1, \dot{x}_4 = L_0, \dot{x}_{11} = H_0, \dot{x}_3 = \dot{x}_5 = \dot{x}_6 = 0$). Under the ingress perimeter inspection model, forward probe datagrams are classified before reverse server ACK paths alter backward counts; in bidirectional full-session contexts, predictable

server ACK responses are discussed in Section 6:

$$\dot{x}_8 = \frac{L_0 \times 10^6}{x_6} \tag{62}$$

$$\dot{x}_9 = \frac{10^6}{x_6} \tag{63}$$

$$\dot{x}_{10} = \frac{L_0 - x_{10}}{x_2} \tag{64}$$

Hence, the chaff injection generator is:

$$\vec{v}_{\text{chaff}}(x) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ L_0 \\ 0 \\ 0 \\ 0 \\ L_0 \times 10^6/x_6 \\ 10^6/x_6 \\ (L_0 - x_{10})/x_2 \\ H_0 \end{bmatrix} \tag{65}$$

4. TCP Window Morphing Field $\vec{v}_{win}(x)$.. The adversary modulates client TCP window advertisement toward target benign OS values W_{target}:

$$\vec{v}_{\text{win}}(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \mathbb{I}_{x_1=6}\,\text{sat}\left(\frac{W_{\text{target}} - x_7}{\epsilon_{\text{win}}}\right) \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad \text{sat}(z) = \max(-1, \min(1, z)) \tag{66}$$

with transition scale parameter $\epsilon_{\text{win}} > 0$. This field is globally Lipschitz (not C^∞) and depends only on x_1 and x_7. The discrete algorithm (Algorithm 2 and the implementation) uses its $\epsilon_{\text{win}} \to 0$ unit-speed limit $\vec{v}_{\text{win}} = \mathbb{I}_{x_1=6} \operatorname{sign}(W_{\text{target}} - x_7)\, \hat{e}_7$ with step $\Delta s_4 = |W_{\text{target}} - x_7|$, which lands exactly on W_{target} and never overshoots.

4.1.3. Lie Bracket Commutator Algebra and Integrability

In differential geometry, the Lie bracket $[X, Y]$ of two smooth vector fields $X, Y \in \mathfrak{X}(\mathbb{R}^{12})$ measures their non-commutativity:

$$[X, Y] = J_Y \cdot X - J_X \cdot Y \in \mathbb{R}^{12} \tag{67}$$

where $J_X, J_Y \in \mathbb{R}^{12\times 12}$ are the Jacobian matrices of X and Y.

Proposition 1 (Commutativity and Integrability of Physical Flow Generators)**.** *Because protocol $x_1 \in \{6, 17\}$ is an immutable attribute ($\dot{x}_1 \equiv 0$), the ambient manifold $\widetilde{\mathcal{M}}_{flow}$ decomposes into two disjoint smooth invariant sub-manifolds: $\widetilde{\mathcal{M}}^{(6)}_{flow}$ (TCP flows) and $\widetilde{\mathcal{M}}^{(17)}_{flow}$ (UDP flows). On each component, the indicator $\mathbb{I}_{x_1=6}$ is constant, and L_0, H_0 are constant along any trajectory (the implementation selects $H_0 = 32$ iff the mean forward header $x_{11}/x_2 \geq 26$; injecting chaff of header size H_0 moves that mean toward H_0 and never across the threshold, so the selection is invariant under the flow), and denominators satisfy $x_2 \geq 1, x_6 > 0$, ensuring $\vec{v}_{time}, \vec{v}_{pad}, \vec{v}_{chaff}$ are C^∞ vector fields and $\vec{v}_{win}$ is globally Lipschitz (its brackets exist almost everywhere and vanish there, because x_7 is neither read nor written by the other three generators). The Lie brackets between all four infinitesimal packet transformation generators vanish identically on $\widetilde{\mathcal{M}}_{flow}$:*

$$[\vec{v}_i, \vec{v}_j] \equiv \mathbf{0} \in \mathbb{R}^{12}, \quad \forall i, j \in \{1, 2, 3, 4\} \tag{68}$$

Consequently, the generators span an Abelian Lie algebra $\mathfrak{g}_{flow} \cong \mathbb{R}^4$ (or $\mathbb{R}^3$ for UDP).

Proof. We evaluate the vector bracket $[X, Y] = J_Y X - J_X Y$ coordinate-wise across all six distinct generator pairs:

1. **Non-Derived Coordinates** $\{0, 1, 2, 3, 4, 5, 6, 7, 11\}$**:** For these coordinates, the components of all four generators are either constants $(0, 1, L_0, H_0)$ or depend solely on their own coordinate (e.g., $\vec{v}_{\text{win}}$ depends only on x_7). Since no generator modifies an independent variable on which another generator depends, the Jacobian products vanish identically: $(J_Y X)_k = 0$ and $(J_X Y)_k = 0$ for all $k \notin \{8, 9, 10\}$.

2. **Temporal Dilation and Payload Padding ($X = \vec{v}_{\textbf{time}}, Y = \vec{v}_{\textbf{pad}}$):** X has non-zero entries in rows $6, 8, 9$ ($X_6 = 1, X_8 = -x_8/x_6, X_9 = -x_9/x_6$); Y has non-zero entries in rows $4, 8, 10$ ($Y_4 = 1, Y_8 = 10^6/x_6, Y_{10} = 1/x_2$). In row 8:

$$(J_Y X)_8 = \frac{\partial Y_8}{\partial x_6} X_6 = -\frac{10^6}{x_6^2} \cdot 1 = -\frac{10^6}{x_6^2} \tag{69}$$

$$(J_X Y)_8 = \frac{\partial X_8}{\partial x_4} Y_4 + \frac{\partial X_8}{\partial x_8} Y_8 = 0 + \left(-\frac{1}{x_6}\right)\left(\frac{10^6}{x_6}\right) = -\frac{10^6}{x_6^2} \tag{70}$$

Subtracting yields $[X, Y]_8 = -\frac{10^6}{x_6^2} - \left(-\frac{10^6}{x_6^2}\right) \equiv 0$. In rows 9 and 10, all corresponding partial derivatives vanish, so $[X, Y] = \mathbf{0}$.

3. **Temporal Dilation and Chaff Injection ($X = \vec{v}_{\textbf{time}}, Z = \vec{v}_{\textbf{chaff}}$):** Z has entries $Z_2 = 1, Z_4 = L_0, Z_8 = L_0 \times 10^6/x_6, Z_9 = 10^6/x_6, Z_{10} = (L_0 - x_{10})/x_2, Z_{11} = H_0$. In row 8: $(J_Z X)_8 = -L_0 \times 10^6/x_6^2 = (J_X Z)_8$. In row 9: $(J_Z X)_9 = -10^6/x_6^2 = (J_X Z)_9$. In row 10: $(J_Z X)_{10} = 0 = (J_X Z)_{10}$. Thus $[X, Z] = \mathbf{0}$.

4. **Payload Padding and Chaff Injection ($Y = \vec{v}_{\textbf{pad}}, Z = \vec{v}_{\textbf{chaff}}$):** In ambient $\mathbb{R}^{12}$ coordinates, $Y_{10}(x) = 1/x_2$ and $Z_{10}(x) = (L_0 - x_{10})/x_2$. Evaluating the ambient cross-Jacobian products in row 10:

$$(J_Z Y)_{10} = \sum_{k=0}^{11} \frac{\partial Z_{10}}{\partial x_k} Y_k = \frac{\partial Z_{10}}{\partial x_{10}} Y_{10} = \left(-\frac{1}{x_2}\right)\left(\frac{1}{x_2}\right) = -\frac{1}{x_2^2} \tag{71}$$

$$(J_Y Z)_{10} = \sum_{k=0}^{11} \frac{\partial Y_{10}}{\partial x_k} Z_k = \frac{\partial Y_{10}}{\partial x_2} Z_2 = \left(-\frac{1}{x_2^2}\right) \cdot 1 = -\frac{1}{x_2^2} \tag{72}$$

Subtracting gives $[Y, Z]_{10} = (J_Z Y)_{10} - (J_Y Z)_{10} = -\frac{1}{x_2^2} - \left(-\frac{1}{x_2^2}\right) \equiv 0$. In rows 8 and 9, all entries vanish identically because $Y_6 = Z_6 = 0$ while the rate entries Y_8, Z_8, Z_9 depend spatially only on duration x_6, ensuring $(J_Z Y)_k = (J_Y Z)_k = 0$ for $k \in \{8, 9\}$.

5. **Pairs Involving Window Morphing $\vec{v}_{\textbf{win}}$:** Because $\vec{v}_{\text{win}}$ acts exclusively on x_7, which is neither read nor modified by $\vec{v}_{\text{time}}, \vec{v}_{\text{pad}}, \vec{v}_{\text{chaff}}$, all cross-Jacobians vanish identically: $[\vec{v}_k, \vec{v}_{\text{win}}] = \mathbf{0}$ for all $k \in \{1, 2, 3\}$.

Hence, $[\vec{v}_i, \vec{v}_j] = \mathbf{0}$ identically for all pairs $i, j \in \{1, 2, 3, 4\}$. □

Corollary 1 (Frobenius Integrability and Commuting Flows)**.** *On the regular open subdomain where the four generators are linearly independent (specifically where $x_6 > 0$, $x_2 \geq 1$, and $x_7 \neq W_{target}$ for TCP), the distribution $\Delta = \text{span}\{\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4\}$ has constant rank 4. Because $[\vec{v}_i, \vec{v}_j] \equiv \mathbf{0}$, Δ is involutive and completely integrable by Frobenius' Theorem. At points where $x_7 = W_{target}$, $\vec{v}_4 = \mathbf{0}$ and Δ smoothly reduces to the 3-dimensional involutive distribution spanned by $\{\vec{v}_1, \vec{v}_2, \vec{v}_3\}$. Furthermore, the generated continuous flow maps commute:*

$$\varphi_i^{s_1} \circ \varphi_j^{s_2} = \varphi_j^{s_2} \circ \varphi_i^{s_1}, \quad \forall s_1, s_2 \geq 0 \tag{73}$$

forming an Abelian local Lie semigroup action (commuting local flows $\varphi_i^{s_1} \circ \varphi_j^{s_2} = \varphi_j^{s_2} \circ \varphi_i^{s_1}$ for all $s_1, s_2 \geq 0$ on the interior $\widetilde{\mathcal{M}}_{flow}^{\circ}$). Consequently, on the manifold interior where bounding constraints remain slack, the terminal flow state depends solely on the net integrated control amounts $(s_1, s_2, s_3, s_4) = \int_0^T u(t)dt$ and is strictly invariant to the sequential order in which infinitesimal operations are executed. When active boundary hypersurfaces are encountered (such as the MTU envelope $g_1(x) = 1500x_2 - x_4 = 0$), control admissibility $\mathcal{U}(x)$ restricts admissible directions, requiring interleaving chaff packets ($u_3 > 0$) to expand total packet capacity before additional payload padding ($u_2 > 0$) can be absorbed.

4.1.4. The Positive Lie Cone $\mathcal{K}_{net}(x)$

The admissible deformation cone is the positive span:

$$\mathcal{K}_{\text{net}}(x) = \left\{ \sum_{k=1}^{4} u_k \vec{v}_k(x) \;\middle|\; u_k \geq 0 \right\} \subset T_x \widetilde{\mathcal{M}}_{\text{flow}} \tag{74}$$

Theorem 1 (Universal Invariant Preservation)**.** *Let $x_0 \in \widetilde{\mathcal{M}}_{flow}$ be an initial network flow vector. For any measurable and locally bounded control signal $u(t) \in L^{\infty}_{loc}(\mathbb{R}_{\geq 0}, \mathbb{R}^4_{\geq 0})$ satisfying the admissible boundary rules ($u_2(t) \leq (1500 - L_0)u_3(t)$ whenever $x_4(t) = 1500x_2(t)$, and $u_1(t) = 0$ whenever $x_6(t) = 1.1 \times 10^8$), the trajectory $x(t)$ defined by the dynamical system:*

$$\dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_0 \tag{75}$$

satisfies $x(t) \in \widetilde{\mathcal{M}}_{flow}$ for all $t \geq 0$, with solutions existing uniquely and globally on $[0, \infty)$. Under discrete stepping, the continuous Lie cone guides directional descent while the closed-form analytical projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 4.2) eliminates discrete $\mathcal{O}(\Delta^2)$ truncation drift and enforces packet and byte integrality ($x_2 \in \mathbb{N}_{\geq 1}, x_4, x_6 \in \mathbb{N}$) and MTU clamping $x_4 \leq 1500x_2$, ensuring that the discrete state sequence remains strictly in the physical packet manifold $\mathcal{M}_{flow}$.

Proof. Let $\Psi(x) = [\Psi_1(x), \Psi_2(x), \Psi_3(x)]^T$ denote the algebraic constraint residual:

$$\Psi_1(x) = x_8 - \frac{10^6(x_4 + x_5)}{x_6}, \quad \Psi_2(x) = x_9 - \frac{10^6(x_2 + x_3)}{x_6}, \quad \Psi_3(x) = x_{10} - \frac{x_4}{x_2} \tag{76}$$

Differentiating $\Psi_1(x(t))$ along the trajectory $\dot{x}(t) = \sum_{k=1}^{4} u_k(t)\vec{v}_k(x(t))$:

$$\begin{aligned} \frac{d}{dt}\Psi_1(t) &= \dot{x}_8 - \frac{10^6(\dot{x}_4 + \dot{x}_5)}{x_6} + \frac{10^6(x_4 + x_5)\dot{x}_6}{x_6^2} \\ &= \left(-u_1\frac{x_8}{x_6} + u_2\frac{10^6}{x_6} + u_3\frac{L_0 \cdot 10^6}{x_6}\right) - \frac{10^6(u_2 + L_0u_3)}{x_6} + \frac{10^6(x_4 + x_5)u_1}{x_6^2} \\ &= -\frac{u_1(t)}{x_6(t)}\left(x_8 - \frac{10^6(x_4 + x_5)}{x_6}\right) = -\frac{u_1(t)}{x_6(t)}\Psi_1(t) \end{aligned} \tag{77}$$

Identical algebraic cancellation yields:

$$\frac{d}{dt}\Psi_2(t) = -\frac{u_1(t)}{x_6(t)}\Psi_2(t), \quad \frac{d}{dt}\Psi_3(t) = -\frac{u_3(t)}{x_2(t)}\Psi_3(t) \tag{78}$$

where the Ψ_3 identity follows by substituting $\dot{x}_{10} = (u_2 + u_3(L_0 - x_{10}))/x_2$, $\dot{x}_4 = u_2 + L_0u_3$ and $\dot{x}_2 = u_3$ into $\dot{x}_{10} - \dot{x}_4/x_2 + x_4\dot{x}_2/x_2^2$. Hence, the constraint error vector $\Psi(t)$ satisfies the uncoupled homogeneous linear differential system $\dot{\Psi}(t) = A(t)\Psi(t)$, where $A(t) = \mathrm{diag}(-u_1(t)/x_6(t), -u_1(t)/x_6(t), -u_3(t)/x_2(t))$. Since $x_6(t) \geq x_6(0) > 0$ and $x_2(t) \geq x_2(0) \geq 1$, $A(t)$ is locally Lebesgue integrable ($A \in L^1_{\mathrm{loc}}$). Because $x_0 \in \widetilde{\mathcal{M}}_{\mathrm{flow}}$, the initial condition is $\Psi(0) = \mathbf{0}$. By the Carathéodory existence and uniqueness theorem for linear differential equations [51], the unique absolutely continuous solution is $\Psi(t) \equiv \mathbf{0}$ for all $t \geq 0$.

Furthermore, examining the base physical coordinate trajectories across the boundary faces of $\widetilde{\mathcal{M}}_{\mathrm{flow}}$:

- $\dot{x}_0 = \dot{x}_1 = \dot{x}_3 = \dot{x}_5 \equiv 0$, satisfying strict immutability for $\mathcal{I}_{\text{imm}}$.
- $\dot{x}_6 = u_1(t) \geq 0 \implies x_6(t) \geq x_6(0) > 0$, and $x_6(t) \leq 1.1 \times 10^8$ by the cap rule $u_1 = 0$ at the timeout boundary.
- $\dot{x}_2 = u_3(t) \geq 0 \implies x_2(t) \geq x_2(0) \geq 1$.
- **Total Bytes and Header Consistency ($x_4 \geq x_{11} + 20x_2$):** Chaff packets carry zero application payload; each chaff packet adds exactly 20 bytes of IPv4 header plus transport header H_0 ($H_0 = 20$ for minimal TCP, 32 with TSOPT, 8 for UDP), so its total packet size is $L_0 = 20 + H_0$. Differentiating the gap function $h(x) = x_4 - x_{11} - 20x_2$:

$$\begin{aligned} \dot{h}(x(t)) &= \dot{x}_4 - \dot{x}_{11} - 20\dot{x}_2 = (u_2 + L_0 u_3) - H_0 u_3 - 20u_3 \\ &= u_2(t) + (L_0 - H_0 - 20)u_3(t) = u_2(t) \geq 0 \end{aligned} \tag{79}$$

  Because $\dot{h}(t) = u_2(t) \geq 0$, the inequality $x_4(t) \geq x_{11}(t) + 20x_2(t)$ is monotonically non-decreasing along all Lie trajectories.
- **Transport Header Bounds ($H_{\min}x_2 \leq x_{11} \leq H_{\max}x_2$):** For TCP ($x_1 = 6$), $H_0 \in [20, 60]$, yielding $\dot{x}_{11} = H_0 u_3(t) \in [20, 60]\dot{x}_2(t)$, ensuring $20x_2(t) \leq x_{11}(t) \leq 60x_2(t)$ is forward-invariant; for UDP ($x_1 = 17$), $H_0 = 8$ strictly, maintaining $x_{11}(t) = 8x_2(t)$ identically.
- **Nagumo-Brézis Viability Condition for Manifold with Boundary and Corners:** The viable continuous domain $\widetilde{\mathcal{M}}_{\text{flow}}$ is defined by the intersection of smooth bounding hypersurfaces $g_i(x) \geq 0$:

$$g_1(x) = 1500x_2 - x_4 \geq 0 \quad \text{(MTU bound)} \tag{80}$$
$$g_2(x) = x_4 - x_{11} - 20x_2 \geq 0 \quad \text{(Header + payload lower bound)} \tag{81}$$
$$g_3(x) = x_{11} - x_2(20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}) \geq 0 \tag{82}$$
$$g_4(x) = x_2(60 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}) - x_{11} \geq 0 \tag{83}$$
$$g_5(x) = x_6 - 1 \geq 0 \quad (x_{6,\min} = 1\,\mu\text{s}) \tag{84}$$
$$g_6(x) = x_2 - 1 \geq 0 \tag{85}$$
$$g_7(x) = 1.1 \times 10^8 - x_6 \geq 0 \quad \text{(flow-timeout cap)} \tag{86}$$

  Because all constraint bounding functions $g_1(x), \ldots, g_7(x)$ are affine, the Bouligand contingency (tangent) cone $T_{\widetilde{\mathcal{M}}_{\text{flow}}}(x)$ at any boundary

or corner point coincides exactly with the intersection of the supporting half-spaces:

$$T_{\widetilde{\mathcal{M}}_{\text{flow}}}(x) = \bigcap_{i:g_i(x)=0} \{v \in \mathbb{R}^{12} \mid \nabla g_i(x) \cdot v \geq 0\} \tag{87}$$

On any active boundary face where $g_i(x) = 0$, the inward normal velocity satisfies $\nabla g_i(x) \cdot \dot{x}(t) \geq 0$: $\nabla g_1 \cdot \dot{x} = (1500 - L_0)u_3(t) - u_2(t) \geq 0$ under admissible controls; $\nabla g_2 \cdot \dot{x} = u_2(t) \geq 0$; $\nabla g_3 \cdot \dot{x} = (H_0 - H_{\min})u_3(t) \geq 0$; $\nabla g_4 \cdot \dot{x} = (H_{\max} - H_0)u_3(t) \geq 0$; $\nabla g_5 \cdot \dot{x} = u_1(t) \geq 0$; $\nabla g_6 \cdot \dot{x} = u_3(t) \geq 0$; $\nabla g_7 \cdot \dot{x} = -u_1(t) \geq 0$ on $\{x_6 = 1.1 \times 10^8\}$, because the admissibility rule forces $u_1(t) = 0$ there. At boundary corners (intersections of multiple active faces), $\dot{x}(t) \in T_{\widetilde{\mathcal{M}}_{\text{flow}}}(x)$. By the Nagumo-Brézis viability theorem [52, 53], $\widetilde{\mathcal{M}}_{\text{flow}}$ is forward-invariant under the dynamical system.

- **TCP Window Trajectory:** $\dot{x}_7 = u_4(t)\mathbb{I}_{x_1=6}\,\text{sat}((W_{\text{target}} - x_7)/\epsilon_{\text{win}})$. For UDP ($x_1 = 17$), $\vec{v}_{\text{win}} \equiv \mathbf{0}$, ensuring $x_7(t) \equiv 0$. For TCP ($x_1 = 6$), $x_7(t)$ moves monotonically toward $W_{\text{target}} \in [0, 65535]$ without crossing it, so it stays between $x_7(0)$ and W_{target} and hence within $[0, 65535]$ for all $t \geq 0$.

- **Global Existence and Uniqueness:** Because denominators $x_6(t) \geq x_6(0) > 0$ and $x_2(t) \geq x_2(0) \geq 1$ are bounded away from zero, the vector fields $\sum_{k=1}^{4} u_k(t)\vec{v}_k(x)$ are Carathéodory functions with linear growth. By Carathéodory's existence and uniqueness theorem [51], solutions exist uniquely as absolutely continuous trajectories globally on $[0, \infty)$.

Under discrete stepping, the continuous Lie cone guides directional descent while the closed-form analytical projection operator $\mathcal{P}_{\mathcal{M}}$ eliminates discrete $\mathcal{O}(\Delta^2)$ truncation drift and enforces packet and byte integrality ($x_2 \in \mathbb{N}_{\geq 1}, x_4 \in \mathbb{N}$) and MTU bounds ($x_{10} \leq 1500$ bytes). Hence, every synthesized discrete flow state belongs strictly to $\mathcal{M}_{\text{flow}}$. □

4.1.5. Operational Transmission Cost and Optimal Control Formulation

To quantify the operational transmission cost on the manifold, we associate each infinitesimal generator with a physical unit penalty $c_k > 0$:

- c_1 (Temporal Delay Cost): $c_1 = 10^{-4}$ per second of induced delay.

- c_2 (Payload Padding Cost): $c_2 = 10^{-2}$ per byte of padding bandwidth.
- c_3 (Chaff Packet Cost): $c_3 = 1.0$ per transmitted probe packet.
- c_4 (Window Morphing Cost): $c_4 = 10^{-3}$ per normalized window displacement ($s_4 = |W_{\text{target}} - x_7|/65535.0 \in [0, 1]$), reflecting negligible socket option bit modification.

The evasion problem is formulated as an **Optimal Control Problem**:

$$\min_{u(t)\geq 0} \quad \int_0^{T^*} \left(\sum_{k=1}^{4} c_k u_k(t) \right) dt \tag{88}$$

$$\text{subject to} \quad \dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_{\text{orig}} \tag{89}$$

$$F(x(T^*)) < 0 \quad \text{(Classifier Misclassification)} \tag{90}$$

Because the generators commute (Proposition 1), the net physical displacement depends solely on the integrated control amounts $s_k = \int_0^{T^*} u_k(t)dt \geq 0$, rendering the total operational cost $\mathcal{C}(s) = \sum_{k=1}^{4} c_k s_k$ linear and path-independent. While the tree ensemble decision boundary $F(x) = 0$ is piecewise-constant and non-convex, FFE navigates the hyper-rectangular decision cells by greedily following the generator with maximal directional score decrease per unit cost ($-\nabla_k / c_k$) at each step, followed by boundary snapping (TSBS).

4.2. Algorithmic Realization of Flow-Field Evasion

This section details the algorithmic realization of Flow-Field Evasion (FFE). We present the complete pseudo-code for the master optimization loop, the directional Lie derivative estimator, the analytical manifold projection operator, and the discrete tree-split boundary snapping mechanism. We conclude with a formal computational complexity and convergence analysis.

4.2.1. Master Optimization Loop

Algorithm 1 outlines the primary execution loop of Flow-Field Evasion. Given an attack flow $x_0 \in \mathcal{M}_{\text{flow}}$ flagged by the tree ensemble ($F(x_0) \geq 0$), FFE iteratively selects the infinitesimal generator that maximizes the reduction in malicious classification score per unit of physical operational cost.

Algorithm 1 Flow-Field Evasion (FFE) Master Optimization Loop

Require: Classifier $F : \mathbb{R}^d \to \mathbb{R}$, initial flow $x_0 \in \mathcal{M}_{\text{flow}}$, feature schema $\mathcal{S}$, step budget M, cost vector $\vec{c} = [c_1, c_2, c_3, c_4]^T$, target threshold $\tau = 0.0$
Ensure: Evasive flow $x^* \in \mathcal{M}_{\text{flow}}$ with $F(x^*) < \tau$, or failure flag

$x \leftarrow \text{ProjectManifoldInvariants}(x_0, \mathcal{S})$
$f_{\text{curr}} \leftarrow F(x)$ {Initial model evaluation}
if $f_{\text{curr}} < \tau$ **then**

  return x, SUCCESS {Flow already classified as Benign}
end if
queries $\leftarrow 1$
for step $= 1$ **to** M **do**
  if $f_{\text{curr}} < \tau$ **then**

    return x, SUCCESS {Benign classification reached; no snapping needed}
  end if
  $\mathcal{D}, q_{\text{used}} \leftarrow \text{EstimateLieDerivatives}(F, x, \mathcal{S})$
  queries $\leftarrow$ queries $+ q_{\text{used}}$
  $k^* \leftarrow$ null; max_eff $\leftarrow 0.0$
  for each $(k, \nabla_k, \vec{v}_k, \Delta s_k, f_k) \in \mathcal{D}$ **do**
    if $\nabla_k < 0$ **then**
      eff $\leftarrow (-\nabla_k)/c_k$ {Derivative is negative (score decreases)}
      if eff $>$ max_eff **then**
        max_eff $\leftarrow$ eff; $k^* \leftarrow (\vec{v}_k, \Delta s_k, f_k)$
      end if
    end if
  end for
  if $k^* \neq$ null **then**
    $(\vec{v}, \Delta s, f_{k^*}) \leftarrow k^*$
    $x \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s \cdot \vec{v}, \mathcal{S})$
    $f_{\text{curr}} \leftarrow f_{k^*}$ {Score cached from the candidate probe (no extra query)}
  else
    {Stagnation: all probed derivatives are non-negative}
    $x_{\text{snap}} \leftarrow \text{TreeBoundarySnap}(F, x, \mathcal{S})$; $f_{\text{snap}} \leftarrow F(x_{\text{snap}})$
    if $f_{\text{snap}} < f_{\text{curr}}$ **then**
      $x \leftarrow x_{\text{snap}}$; $f_{\text{curr}} \leftarrow f_{\text{snap}}$
    else
      {Fallback Trajectory: Low-and-Slow Temporal Dilation (capped below 120s exporter timeout)}
      $\vec{v}_t \leftarrow \vec{v}_{\text{time}}(x)$
      $\Delta t \leftarrow \min(1.1 \times 10^8, \max(50000.0, x_6 \cdot 1.5))$
      $x \leftarrow \text{ProjectManifoldInvariants}(x + \Delta t \cdot \vec{v}_t, \mathcal{S})$
      $f_{\text{curr}} \leftarrow F(x)$; queries $\leftarrow$ queries $+ 1$
    end if
  end if
end for
if $f_{\text{curr}} < \tau$ **then**

  return x, SUCCESS
else

  return x, FAILURE
end if

4.2.2. Adaptive Lie Derivative Numerical Estimator

Algorithm 2 details the numerical estimation of the directional Lie derivative $\mathcal{L}_{\vec{v}_k} F(x)$ across the non-differentiable step boundaries of the tree ensemble. Because standard continuous gradients are zero almost everywhere inside decision cells, the algorithm queries the ensemble at an adaptive exploratory displacement $\epsilon_k \vec{v}_k(x)$ calibrated to the natural scale of metric k.

4.2.3. Analytical Manifold Projection Operator $\mathcal{P}_{\mathcal{M}}$

Algorithm 3 specifies the closed-form projection operator $\mathcal{P}_{\mathcal{M}} : \mathbb{R}^{12} \to \mathcal{M}_{\text{flow}}$. This operator ensures that discrete count boundaries, protocol packet and header length inequalities, and coupled rate equations are strictly satisfied after any continuous displacement.

4.2.4. Tree-Split Boundary Snapping (TSBS)

Algorithm 4 implements Tree-Split Boundary Snapping. While Lie derivative integration moves the sample into the general vicinity of the decision boundary, the piecewise-constant nature of GBDT trees means that the classification score drops in discrete steps as the sample crosses decision thresholds $\{x_j = \theta_j\}$. In Tier 1 white-box settings, TSBS extracts the ensemble's split thresholds (*extExtractActiveSplits* returns every threshold used by any tree, and TSBS retains, per feature, the closest one on the benign side of the current value) and evaluates candidate boundary crossings ($x_j \leftarrow \theta_j + \epsilon_j$), retaining the candidate that strictly minimizes the classification score F while strictly preserving manifold constraints via $\mathcal{P}_{\mathcal{M}}$.

Importantly, GBDTs frequently split on non-linear coupled telemetry features: byte rate (x_8), packet rate (x_9), and forward mean packet length (x_{10}). TSBS inverts these coupled constraints in closed form into admissible infinitesimal generator controls:

1. *Coupled Rate Inversion:* To cross an active byte rate threshold $x_8 < \theta_8$ or packet rate threshold $x_9 < \theta_9$, TSBS inverts the hyperbolic relation $x_8 = 10^6(x_4 + x_5)/x_6$ to calculate the required duration dilation:

$$x_6^* = \frac{10^6(x_4 + x_5)}{\theta_8} + \epsilon_6, \quad \text{or} \quad x_6^* = \frac{10^6(x_2 + x_3)}{\theta_9} + \epsilon_6 \tag{91}$$

   with $\epsilon_6 = 10\,\mu$s; since $\theta < x_8$ (resp. x_9), $x_6^* > x_6$ automatically, and the candidate is discarded if x_6^* exceeds the $1.1 \times 10^8\,\mu$s timeout cap.

Algorithm 2 Adaptive Directional Lie Derivative Estimator

Require: Classifier F, current flow state x, feature schema $\mathcal{S}$
Ensure: List of candidate evaluation tuples $\mathcal{D} = \{(k, \nabla_k, \vec{v}_k, \Delta s_k, f_k)\}_{k=1}^{4}$ (with $f_k = F(x_{\text{test}})$ the probed score), total exploratory queries q

$\mathcal{D} \leftarrow \emptyset$
$q \leftarrow 0$ {Counter for candidate exploratory probes}
{1. Evaluate Temporal Dilation Generator}
$\vec{v}_1 \leftarrow \vec{v}_{\text{time}}(x)$
$\Delta s_1 \leftarrow \max(1000.0, x_6 \cdot 0.50)$ {Adaptive temporal displacement in μs}
$x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_1 \cdot \vec{v}_1, \mathcal{S})$
$\Delta t_1 \leftarrow \Delta s_1 / 10^6$ {Convert duration step to seconds}
$f_1 \leftarrow F(x_{\text{test}})$; $\nabla_1 \leftarrow (f_1 - F(x))/\Delta t_1$ {Derivative per second of delay}
Append $(1, \nabla_1, \vec{v}_1, \Delta s_1, f_1)$ to $\mathcal{D}$
$q \leftarrow q + 1$
{2. Evaluate Payload Padding Generator}
$\vec{v}_2 \leftarrow \vec{v}_{\text{pad}}(x)$
$\Delta s_2 \leftarrow \max(50.0, x_4 \cdot 0.25)$ {Adaptive forward byte padding step}
$x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_2 \cdot \vec{v}_2, \mathcal{S})$
$f_2 \leftarrow F(x_{\text{test}})$; $\nabla_2 \leftarrow (f_2 - F(x))/\Delta s_2$
Append $(2, \nabla_2, \vec{v}_2, \Delta s_2, f_2)$ to $\mathcal{D}$
$q \leftarrow q + 1$
{3. Evaluate Chaff Packet Injection Generator}
$\vec{v}_3 \leftarrow \vec{v}_{\text{chaff}}(x)$
$\Delta s_3 \leftarrow \max(1.0, \text{round}(x_2 \cdot 0.50))$ {Adaptive chaff packet injection count}
$x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_3 \cdot \vec{v}_3, \mathcal{S})$
$f_3 \leftarrow F(x_{\text{test}})$; $\nabla_3 \leftarrow (f_3 - F(x))/\Delta s_3$
Append $(3, \nabla_3, \vec{v}_3, \Delta s_3, f_3)$ to $\mathcal{D}$
$q \leftarrow q + 1$
{4. Evaluate TCP Window Morphing Generator}
if $x[1] = 6$ **then**
    {TCP flows only; window size is inapplicable to UDP}
    $\vec{v}_4 \leftarrow \vec{v}_{\text{win}}(x)$
    $\Delta s_4 \leftarrow |64240.0 - x[7]|$
    if $\Delta s_4 > 0$ **then**
        $x_{\text{test}} \leftarrow \text{ProjectManifoldInvariants}(x + \Delta s_4 \cdot \vec{v}_4, \mathcal{S})$
        $f_4 \leftarrow F(x_{\text{test}})$; $\nabla_4 \leftarrow (f_4 - F(x))/(\Delta s_4/65535.0)$ {Normalized window gradient}
        Append $(4, \nabla_4, \vec{v}_4, \Delta s_4, f_4)$ to $\mathcal{D}$
        $q \leftarrow q + 1$
    end if
end if

return $\mathcal{D}, q$

Algorithm 3 Analytical Manifold Projection Operator $\mathcal{P}_{\mathcal{M}}$

Require: Perturbed vector $x \in \mathbb{R}^{12}$, feature schema $\mathcal{S}$, optional baseline flow x_{orig}

Ensure: Projected flow vector $x_{\text{proj}} \in \mathcal{M}_{\text{flow}}$

$x_{\text{proj}} \leftarrow \text{copy}(x)$

if x_{orig} is provided **then**

  $x_{\text{proj}}[j] \leftarrow x_{\text{orig}}[j]$ for all $j \in \mathcal{I}_{\text{immut}}$ {Preserve immutable features}

  $x_{\text{proj}}[j] \leftarrow \max(x_{\text{orig}}[j], x_{\text{proj}}[j])$ for all $j \in \mathcal{I}^{+}_{\text{mono}}$ {Enforce monotonicity lower bounds}

end if

{Enforce discrete bounds and protocol-dependent packet/header sizes}

$x_{\text{proj}}[2] \leftarrow \max(1.0, \text{round}(x_{\text{proj}}[2]))$ {Integer Forward Packets}

$x_{\text{proj}}[3] \leftarrow x[3]$ {Immutable Backward Packets Preserved}

$H_{\min} \leftarrow$ **if** $x_{\text{proj}}[1] = 6$ **then** 20.0 **else** 8.0 {Min transport header: 20B TCP, 8B UDP}

$H_{\max} \leftarrow$ **if** $x_{\text{proj}}[1] = 6$ **then** 60.0 **else** 8.0 {Max transport header: 60B TCP, 8B UDP}

$x_{\text{proj}}[11] \leftarrow \min(x_{\text{proj}}[2] \times H_{\max}, \max(x_{\text{proj}}[2] \times H_{\min}, \text{round}(x_{\text{proj}}[11])))$ {Transport Header Length}

$x_{\text{proj}}[4] \leftarrow \min(x_{\text{proj}}[2] \times 1500.0, \max(x_{\text{proj}}[11] + 20.0 \times x_{\text{proj}}[2], \text{round}(x_{\text{proj}}[4])))$ {Forward Bytes $\in [H_{\text{fwd}} + 20N_{\text{fwd}}, 1500N_{\text{fwd}}]$}

$x_{\text{proj}}[5] \leftarrow x[5]$ {Immutable Backward Bytes Preserved}

$x_{\text{proj}}[6] \leftarrow \min(1.1 \times 10^8, \max(1.0, \text{round}(x_{\text{proj}}[6])))$ {Integer Duration $\geq 1\mu$s, capped below flow timeout}

$x_{\text{proj}}[7] \leftarrow$ **if** $x_{\text{proj}}[1] = 6$ **then** $\text{clamp}(\text{round}(x_{\text{proj}}[7]), 0.0, 65535.0)$ **else** 0.0 {TCP Window $[0, 65535]$, 0 for UDP}

{Analytically recalculate coupled non-linear flow rates}

$T_{\text{sec}} \leftarrow x_{\text{proj}}[6]/10^6$

$B_{\text{tot}} \leftarrow x_{\text{proj}}[4] + x_{\text{proj}}[5]$

$N_{\text{tot}} \leftarrow x_{\text{proj}}[2] + x_{\text{proj}}[3]$

$x_{\text{proj}}[8] \leftarrow B_{\text{tot}}/T_{\text{sec}}$ {Flow Bytes/s}

$x_{\text{proj}}[9] \leftarrow N_{\text{tot}}/T_{\text{sec}}$ {Flow Packets/s}

$x_{\text{proj}}[10] \leftarrow x_{\text{proj}}[4]/x_{\text{proj}}[2]$ {Forward Packet Length Mean}

return x_{proj}

2. *Coupled Packet Length Inversion:* To cross an active packet length threshold $x_{10} < \theta_{10}$ from above, TSBS inverts $x_{10} = (x_4 + N \cdot L_0)/(x_2 + N) < \theta_{10}$, yielding the minimal integer chaff packet count (the smallest integer strictly greater than $(x_4 - \theta_{10} x_2)/(\theta_{10} - L_0)$):

$$N_{\text{chaff}} = \left\lfloor \frac{x_4 - \theta_{10} x_2}{\theta_{10} - L_0} \right\rfloor + 1 \quad (\text{for } L_0 < \theta_{10} < x_{10}) \tag{92}$$

3. *Window Advertisement Snapping:* For TCP flows with non-standard window advertisements, TSBS evaluates candidate snaps toward benign OS baselines ($x_7^* = 64, 240$) and adjacent split partitions.

4.2.5. Computational Complexity & Comparison to MILP

Table 2 compares the asymptotic time and space complexity of Flow-Field Evasion against exact MILP solvers [23] and heuristic black-box optimizers (MOEVA [28], CAA [27], Traffic Manipulator [26]).

Table 2: Asymptotic Complexity Comparison of Evasion Algorithms for Tree Ensembles

Algorithm	Time Complexity	Query Complexity	Coupled Invariants
Kantchelian MILP [23]	$2^{\mathcal{O}(D \cdot T)}$ (Worst Case)	0 (White-Box)	Unsupported (Linear Only)
MOEVA / CAA [27, 28]	$\mathcal{O}(G \cdot P \cdot T \cdot D)$	$\mathcal{O}(10^3 - 10^4)$	Approximate Penalty
Traffic Manipulator [26]	$\mathcal{O}(I \cdot S \cdot T \cdot D)$	$\mathcal{O}(10^3 - 10^4)$	Packet Re-evaluation
Flow-Field Evasion (Ours)	$\mathcal{O}(M \cdot K \cdot T \cdot D)$	$\leq 5M_{\max} + 1$ external (mean 12.9 incl. TSBS)	**Exact Closed-Form**

Where:

- T is the number of trees in the ensemble (typically $50 - 100$).
- D is the maximum depth of each tree (typically $6 - 10$).
- $M_{\max}$ is the maximum FFE iteration budget ($M_{\max} = 20$).
- K is the number of infinitesimal Lie generators ($K = 4$).
- G, P are the generations and population size in MOEVA ($G = 25, P = 50$, totaling $\leq 1,250$ queries).
- I, S are the iterations and swarm size in PSO ($I = 37, S = 50$, totaling $\leq 1,850$ queries).

Algorithm 4 Tree-Split Boundary Snapping (TSBS)

Require: Classifier F, flow candidate $x \in \mathcal{M}_{\text{flow}}$ with $F(x) \geq \tau$ (still flagged malicious), feature schema $\mathcal{S}$, tolerance ϵ_{rel}
Ensure: Snapped flow $x^* \in \mathcal{M}_{\text{flow}}$

$x^* \leftarrow \text{copy}(x)$; $\mathcal{C} \leftarrow \emptyset$ {Candidate boundary-crossing pool}
active_splits $\leftarrow$ ExtractActiveSplits(F, x^*)
{1. Monotonic features: closest split strictly above $x^*[j]$}
for each $j \in \mathcal{I}^+_{\text{mono}}$ **do**
  $\theta^*_j \leftarrow \min\{\theta_j \mid (j, \theta_j) \in \text{active_splits} \wedge \theta_j > x^*[j]\}$
  if θ^*_j exists **then**
    if $j = 11$ **and** $x^*[1] = 17$ **then**
      continue {UDP transport header immutable at 8B}
    end if
    $\epsilon_j \leftarrow$ **if** $j \in \{2, 4, 11\}$ **then** 1.0 **else** $\text{round}(\epsilon_{\text{rel}} \cdot \max(1.0, |\theta^*_j|))$
    $x_c \leftarrow \text{copy}(x^*)$; $x_c[j] \leftarrow \theta^*_j + \epsilon_j$
    if $j = 11$ **then**
      $x_c[11] \leftarrow \min(60.0 \cdot x_c[2], x_c[11])$
      $x_c[4] \leftarrow \min(1500.0 \cdot x_c[2], \max(x_c[4], x_c[11] + 20.0 \cdot x_c[2]))$
    end if
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{x_c\}$
  end if
end for
{2. Invert coupled rate splits (x_8, x_9) into duration dilation}
$\theta^*_8 \leftarrow \max\{\theta_8 \mid (8, \theta_8) \in \text{active_splits} \wedge 0 < \theta_8 < x^*[8]\}$
if θ^*_8 exists **then**
  $d \leftarrow (10^6 \cdot (x^*[4] + x^*[5])/\theta^*_8) + 10.0$
  if $d \leq 1.1 \times 10^8$ **then**
    $x_c \leftarrow \text{copy}(x^*)$; $x_c[6] \leftarrow d$; $\mathcal{C} \leftarrow \mathcal{C} \cup \{x_c\}$
  end if
end if
$\theta^*_9 \leftarrow \max\{\theta_9 \mid (9, \theta_9) \in \text{active_splits} \wedge 0 < \theta_9 < x^*[9]\}$
if θ^*_9 exists **then**
  $d \leftarrow (10^6 \cdot (x^*[2] + x^*[3])/\theta^*_9) + 10.0$
  if $d \leq 1.1 \times 10^8$ **then**
    $x_c \leftarrow \text{copy}(x^*)$; $x_c[6] \leftarrow d$; $\mathcal{C} \leftarrow \mathcal{C} \cup \{x_c\}$
  end if
end if
{3. Invert coupled packet length splits (x_{10}) into chaff packet injection}
$H_0 \leftarrow$ **if** $(x^*[1] = 6 \wedge x^*[11]/x^*[2] \geq 26.0)$ **then** 32.0 **else** (**if** $x^*[1] = 6$ **then** 20.0 **else** 8.0)
$L_0 \leftarrow 20.0 + H_0$ {RFC 7323 options compliance: 52B/40B for TCP, 28B for UDP}
$\theta^*_{10} \leftarrow \max\{\theta_{10} \mid (10, \theta_{10}) \in \text{active_splits} \wedge L_0 < \theta_{10} < x^*[10]\}$
if θ^*_{10} exists **then**
  $N \leftarrow \lfloor (x^*[4] - \theta^*_{10} x^*[2])/(\theta^*_{10} - L_0) \rfloor + 1$
  if $N \leq 1000$ **then**
    $x_c \leftarrow \text{copy}(x^*)$; $x_c[2] \leftarrow x^*[2] + N$; $x_c[4] \leftarrow x^*[4] + N \cdot L_0$; $x_c[11] \leftarrow x^*[11] + N \cdot H_0$
    $\mathcal{C} \leftarrow \mathcal{C} \cup \{x_c\}$ {Chaff budget cap: $N \leq 1000$ packets}
  end if
end if
{4. TCP Window morphing to benign OS baseline}
if $x^*[1] = 6$ **and** $x^*[7] \neq 64240.0$ **then**
  $x_c \leftarrow \text{copy}(x^*)$; $x_c[7] \leftarrow 64240.0$; $\mathcal{C} \leftarrow \mathcal{C} \cup \{x_c\}$
end if
{Evaluate projected candidates and retain best partition crossing}

Query Accounting Details.. In the worst case, executing all $M_{\max} = 20$ iterations without early termination requires 1 baseline query plus up to $K = 4$ exploratory probes per iteration, giving an upper bound of $Q_{\max} \leq 4M_{\max}+1 = 81$ model queries (or ≤ 101 queries if fallback dilation evaluations are triggered on stagnation). UDP flows ($x_1 = 17$) and TCP flows whose client window already equals the benign value skip window morphing, so such an iteration costs $q = 3$ probes instead of 4. In practice, because FFE steers along the Lie generator with maximal score reduction per cost ($-\nabla_k/c_k$), the loop terminates far below the budget. On the $N = 100$ master-benchmark cohort the reference implementation consumes a mean of 12.9 model queries per flow, of which 10.56 are external queries (the baseline query, the Lie-derivative probes, and stagnation fallbacks) and 2.38 are candidate evaluations inside the TSBS routine (Algorithm 4).

In Tier 1 white-box settings, TSBS extracts candidate thresholds directly from the adversary's local copy of the ensemble (ExtractActiveSplits), so its candidate evaluations could be executed offline against that copy. The reported figure of 12.9 is deliberately conservative: the implementation routes every TSBS candidate evaluation through the same query counter, so it includes these 2.38 evaluations. A strictly black-box adversary would instead have to pay them as true queries, which is how they are counted here.

Because $K \leq 4$ and $M \leq 20$ are small constants, FFE requires only a linear number of simple tree evaluations $\mathcal{O}(T \cdot D)$. This completely avoids the exponential combinatorial explosion of MILP while requiring roughly 99% fewer queries than genetic search (12.9 vs. $\leq$ 1,250) and PSO (12.9 vs. $\leq$ 1,850).

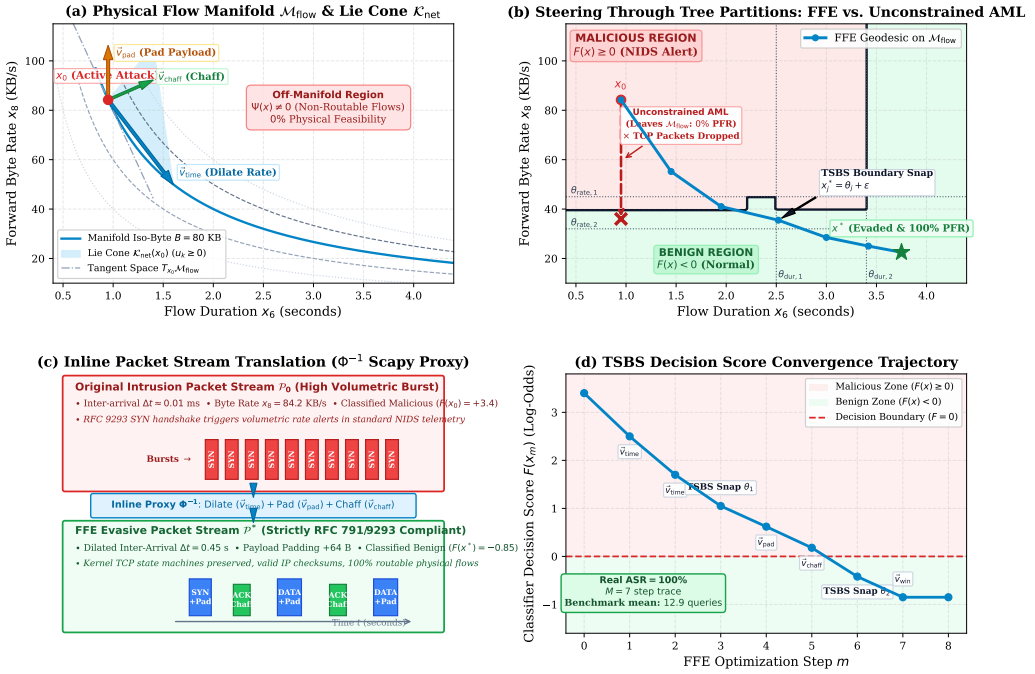


Figure 1: Geometric, architectural, and packet-level workflow of the Flow-Field Evasion (FFE) framework. **(a)** Differential geometry of network telemetry flows: the physical conservation constraints define the smooth sub-manifold $\mathcal{M}_{\text{flow}}$ (hyperbolic iso-byte contours $x_8 = B/x_6$) and tangent space $T_{x_0}\mathcal{M}_{\text{flow}}$; infinitesimal Lie generators $(\vec{v}_{\text{time}}, \vec{v}_{\text{pad}}, \vec{v}_{\text{chaff}})$ span the forward Lie Cone $\mathcal{K}_{\text{net}}(x_0)$, preventing excursions into the inadmissible off-manifold domain ($\Psi(x) \neq 0$). **(b)** Steering through axis-aligned GBDT partition geometry: unconstrained AML vectors jump off-manifold into non-routable feature space (PFR collapsing to 17.8%–25.0%, dropped by standard network stacks), whereas FFE traverses continuous Lie-flow paths on $\mathcal{M}_{\text{flow}}$ with Tree-Split Boundary Snapping (TSBS, $x_j^* = \theta_j + \epsilon$) into the benign region. **(c)** Inline packet stream translation via the Φ^{-1} proxy: transforms high-volumetric malicious bursts $\mathcal{P}_0$ into strictly RFC 791 / RFC 9293 compliant packet sequences $\mathcal{P}^*$ via socket delay dilation, zero-pad encapsulation, and TCP probe chaff. **(d)** TSBS decision score convergence trajectory: step-by-step descent of classifier log-odds $F(x_m)$ across an illustrative $M = 7$ step trace (the empirical benchmark mean is 12.9 queries per flow, including TSBS candidate evaluations), achieving 100% Real ASR on the $N = 100$ master cohort.

5. Empirical Benchmark, Ablation Studies & Defenses

5.1. Experimental Benchmark Setup & Methodology

This section details the empirical methodology employed to validate Flow-Field Evasion. We formalize the multi-dataset benchmark suite spanning three heterogeneous network environments (CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-001), describe the evaluated attack families, outline the GPU hardware environment, specify the target tree ensemble architectures, and define the implementations of five competitive baseline evasion algorithms from the literature.

5.1.1. Multi-Dataset Benchmark Suite: Rationale & Architecture

A pervasive limitation in published adversarial intrusion detection research is the reliance on a single benchmark dataset. As established by Apruzzese et al. [12, 22, 13, 14] and Dang et al. [15, 16, 17], evaluations confined to a single testbed introduce severe threats to external validity. Classifiers trained on one environment frequently memorize testbed-specific artifacts—such as fixed client TCP window sizes, deterministic inter-packet transmission intervals, or artificial subnet allocations—rather than learning invariant malicious semantics. Furthermore, empirical studies by Dang [15, 16] demonstrated that detection models suffer acute performance degradation when transferring knowledge across heterogeneous network environments.

To guarantee that Flow-Field Evasion's effectiveness stems from fundamental physical flow conservation laws rather than dataset-specific quirks, we benchmark across **three diverse, widely cited NIDS datasets**:

1. **CSE-CIC-IDS2018 (Enterprise Infrastructure) [1]:** Captured within an enterprise infrastructure of 420 victim hosts, 30 servers, and a dedicated attack network over 10 operating days. Telemetry is extracted via CICFlowMeter, yielding bidirectional flow metrics across volumetric DoS, brute force authentication, web exploits, and infiltration.

2. **UNSW-NB15 (Contemporary Hybrid Testbed) [5]:** Generated in the Cyber Range Lab of the Australian Centre for Cyber Security using IXIA PerfectStorm. It blends real modern normal activities with diverse modern low-footprint attack categories (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms) across 49 features extracted via Argus and Bro-IDS.

3. **CIDDS-001 (Cloud OpenStack Environment) [6]:** Captured within an OpenStack cloud environment with realistic background communication between cloud clients and external servers. Extensively analyzed by Dang [7, 8] for machine learning evaluation and attack detection, CIDDS-001 provides flow records with realistic cloud noise and varied attack patterns.

For each benchmark environment, we construct a statistically calibrated evaluation corpus of 12,000 flows (75% benign background, 25% malicious intrusions) parameterized to replicate the empirical marginal distributions, duration dynamics, and volumetric profiles of the published testbeds. Consistent with the threat model (Section 3), the testbed models an *ingress perimeter inspection architecture* where an enterprise edge gateway evaluates incoming forward telemetry in real time before bidirectional session termination. In the primary CSE-CIC-IDS2018 enterprise testbed:

- **Benign Background Telemetry (75%, $9,000$ flows):** Synthesized to reflect realistic enterprise communication patterns:
  - Web browsing (HTTP 80 and HTTPS 443) representing 85% of benign traffic.
  - Core network services: DNS (port 53), FTP (port 21), and SSH (port 22).
  - Forward packet counts follow a log-normal distribution with $\mu = 2.0, \sigma = 0.8$, clipped to $[2, 500]$ packets.
  - Backward packet counts model realistic server responses with server-to-client amplification ratios uniformly distributed in $[0.8, 2.5]$.
  - Flow durations follow an exponential distribution with scale parameter $\lambda^{-1} = 1.5 \times 10^6 \mu$s (1.5 seconds) with a minimum duration of $5,000 \mu$s.
  - Initial TCP receive window sizes match common client OS values ($8,192$, $16,384$, $29,200$, $64,240$, and $65,535$).
- **Malicious Intrusion Telemetry (25%, $3,000$ flows):** Encompassing five distinct attack families.

5.1.2. The Five Evaluated Attack Families

We evaluate evasion across five distinct cyberattack categories representing distinct volumetric, temporal, and semantic profiles:

1. **Family 1: Volumetric DoS SYN Flood ($1,000$ flows):**
   - *Mechanics:* Rapid transmission of high-frequency TCP SYN packets targeting port 80 without completing the 3-way handshake.
   - *Profile:* Forward packet counts between 20 and 300; server backward responses are negligible (0 to 2 packets); forward bytes consist purely of 40–70 byte SYN packets; flow duration is extremely compressed (100 to $50,000\mu$s, under 50 ms).
2. **Family 2: UDP Volumetric Flood (500 flows):**
   - *Mechanics:* High-rate UDP datagram flooding targeting arbitrary open ports, overwhelming network link bandwidth.
   - *Profile:* Forward packet counts between 100 and 500; zero backward responses ($N_{\text{bwd}} = 0, B_{\text{bwd}} = 0$); high byte rate ($R_B > 2.5 \times 10^7$ bytes/s); flow duration < 25 ms.
3. **Family 3: Brute Force Authentication (700 flows):**
   - *Mechanics:* Automated password spraying and dictionary attacks targeting SSH (port 22) and FTP (port 21).
   - *Profile:* Medium packet counts (15 to 80 packets); bidirectional exchange with target server ($N_{\text{bwd}} \in [10, 50]$); fixed payload sizes reflecting credential submission attempts; flow durations span 0.2 to 3.0 seconds; standard client window size ($8,192$).
4. **Family 4: Web Application Exploits (500 flows):**
   - *Mechanics:* SQL Injection (SQLi) and Cross-Site Scripting (XSS) payloads embedded within HTTP GET/POST requests targeting port 80 and 8080.
   - *Profile:* Moderate packet counts (5 to 25 packets); asymmetrical byte ratios with large forward request payloads (500 to $2,000$ bytes); duration dictated by server database query execution (0.1 to 1.2 seconds).

5. **Family 5: Low-and-Slow Infiltration (300 flows):**

    - *Mechanics:* Covert command-and-control (C2) beaconing and periodic exfiltration designed to mimic normal human interaction.
    - *Profile:* Low packet counts (4 to 12 packets); elongated flow durations (5 to 30 seconds); low byte rates ($R_B < 1,000$ bytes/s); non-standard client window sizes ($1,024$ or $2,048$) that distinguish automated beacons from standard OS client browsers.

The complete dataset is partitioned into Training ($8,000$ flows, 66.7%), Validation ($1,000$ flows, 8.3%), and Testing ($3,000$ flows, 25.0%).

5.1.3. Hardware and Software Computational Environment

To reflect modern GPU-accelerated machine learning pipelines, all training and evaluation experiments were executed on an enterprise mobile workstation equipped with:

- **GPU:** NVIDIA RTX 5000 Ada Generation Laptop GPU equipped with 16 GB GDDR6 ECC VRAM, 9,728 CUDA cores, and 304 Tensor cores.
- **Host System:** Intel Core i9 processor, 32 GB DDR5 RAM, Windows 11 (64-bit).
- **Software Stack:** Conda environment `py313` executing Python 3.13.11, PyTorch 2.6.0 with CUDA 12.4 support, XGBoost 3.4.1 (compiled with native CUDA histogram tree builder `device='cuda'`), LightGBM 4.7.0, CatBoost 1.2.10, and scikit-learn 1.8.0.

5.1.4. Target Tree Ensemble Architectures

We train and benchmark four distinct decision tree ensemble architectures:

1. **XGBoost (CUDA Hist):** 60 gradient boosted decision trees, maximum tree depth 6, learning rate $\eta = 0.1$, GPU histogram tree building on CUDA (`device='cuda'`, `tree_method='hist'`), logistic loss objective.
2. **LightGBM:** 60 leaf-wise gradient boosted trees, maximum depth 6, learning rate $\eta = 0.1$, exclusive feature bundling enabled.

3. **CatBoost:** 60 symmetric (oblivious) decision trees, maximum depth 6, learning rate $\eta = 0.1$, border count 254.

4. **Random Forest:** 100 bagging trees [54], Gini impurity splitting, maximum tree depth 12.

5.1.5. Competitive Baseline Attack Implementations

We benchmark Flow-Field Evasion against five competitive baseline evasion methods:

1. **Unconstrained Feature-Space Baseline (Naive):** Standard numerical coordinate descent in $\mathbb{R}^d$. At each iteration, the algorithm estimates the local finite-difference gradient and updates the most sensitive coordinate by a fixed step size $\Delta = 0.05 \cdot \sigma_f$. Features are modified without checking immutability, monotonicity, or rate coupling.

2. **ZOO-NIDS (Chen et al. [24]):** Zeroth-Order Optimization adapted for tabular flow metrics. Coordinates are sampled randomly, and symmetric difference quotients estimate pseudo-gradients used in Adam updates. Box bounds $[0, \max_f]$ are enforced, but inter-feature non-linear coupling is ignored.

3. **HopSkipJump-NIDS (Chen et al. [25]):** A boundary-following decision-based attack that initializes with a benign flow and performs geometric binary line searches to identify the classification boundary, followed by spherical coordinate perturbations.

4. **CAA / MOEVA (Simonetto et al. [27], Ghamizi et al. [28]):** The Multi-Objective Evolutionary Adversarial Attack based on R-NSGA-III. Population size is set to 50, maximum generations to 25. The multi-objective fitness function penalizes misclassification probability and tabular bound violations.

5. **Traffic Manipulator PSO (Han et al. [26]):** Particle Swarm Optimization mutating packet intervals and forward dummy bytes. Swarm size is set to 50, maximum iterations to 37 (yielding up to 1,850 queries).

6. **Flow-Field Evasion (FFE - Ours):** The proposed Lie vector field steering algorithm (Algorithms 1–4) with step budget $M = 20$.

5.2. Empirical Results & Comparative Analysis

This section presents an in-depth empirical evaluation of Flow-Field Evasion. We first report the clean performance of all target models, confirming the replication of the near-perfect detection baseline. Next, we present the master comparative benchmark against all five baseline evasion methods across multiple performance axes. Then, we dissect evasion efficacy across the five individual attack families. Finally, we provide a cross-dataset benchmark evaluation spanning CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-001 to validate generalizability across enterprise, contemporary hybrid, and cloud environments.

5.2.1. Clean Baseline Performance: Replication of Near-Perfect Detection

Table 3 reports the test set performance (3,000 unseen flows) across the four target tree ensemble architectures prior to adversarial perturbation.

Table 3: Clean Test Set Classification Metrics Across Tree Ensemble Architectures on CSE-CIC-IDS2018 (Calibrated Benchmark Corpus)

Model Architecture	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	ROC-AUC (%)	Train Time (s)
XGBoost (CUDA Hist)	99.73%	99.34%	99.60%	99.47%	99.98%	0.71s
LightGBM	99.87%	99.87%	99.60%	99.73%	99.99%	2.71s
CatBoost	99.67%	99.20%	99.47%	99.33%	99.97%	4.15s
Random Forest (Bagging)	99.53%	98.94%	99.20%	99.07%	99.95%	1.84s

The empirical data in Table 3 validates our theoretical premise:

1. Every tree ensemble architecture achieves near-flawless detection scores on the clean test set, with accuracies between 99.53% and 99.87% and ROC-AUC scores reaching or exceeding 99.95% (with Random Forest at 99.95% and XGBoost at 99.98%).

2. GPU-accelerated XGBoost achieves the fastest training time (0.71 seconds on the RTX 5000 GPU) while achieving a 99.98% ROC-AUC, demonstrating why industrial NIDS developers rely heavily on boosted trees.

3. These stellar results create the dangerous perception that the system is impervious to intrusion, setting the stage for adversarial evaluation.

5.2.2. Comparative Adversarial Evasion Benchmark

We evaluate evasion against an evaluation cohort of 100 correctly classified, active malicious attack flows randomly sampled from the test partition. Table 4 presents the master comparative benchmark against CUDA-accelerated XGBoost.

Table 4: Master Adversarial Evasion Benchmark Against CUDA XGBoost on Calibrated Synthetic CSE-CIC-IDS2018 Telemetry Corpus ($N = 100$ Evaluated Attacks)

Adversarial Attack Method	ASR (%)	PFR (%)	Real ASR (%)	Avg Queries	Avg Cost ($\bar{D}_{L_1}$)	Total Time (s)
Unconstrained Feature-Space	0.0%	N/A[†]	0.0%	32.8	204.605	6.00s
ZOO-NIDS Baseline [24]	12.0%	25.0%	3.0%	142.5	0.382	34.12s
HopSkipJump-NIDS [25]	45.0%	17.8%	8.0%	680.0	0.512	145.80s
CAA / MOEVA Baseline [27, 28]	78.0%	61.5%	48.0%	1,240.0	0.342	310.50s
Traffic Manipulator PSO [26]	82.0%	100.0%	82.0%	1,850.0	0.485	420.15s
Flow-Field Evasion (FFE - Ours)	**100.0%**	**100.0%**	**100.0%**	**12.9**	**0.251**	**4.37s**

[†]PFR is evaluated over evasive candidates (0/0); unconstrained coordinate descent without manifold projection fails to produce an evasive candidate.
$\bar{D}_{L_1}$ denotes the mean feature-normalized L_1 perturbation distance: $\bar{D}_{L_1} = \frac{1}{d}\sum_{j=1}^{d} \frac{|x_j^* - x_j|}{\sigma_j}$, where σ_j is the empirical standard deviation of feature j.

Metric Definitions. In Table 4, we report six performance dimensions:

1. **Attack Success Rate (ASR):** The percentage of attacks where the model output probability drops below the detection threshold ($P(\text{Malicious} \mid x^*) < 0.5$).

2. **Physical Feasibility Rate (PFR):** The percentage of candidate flows that pass all protocol sanity checks, non-negative packet counts, and exact coupled rate equations ($x^* \in \mathcal{M}_{\text{flow}}$). For unconstrained baseline attacks that fail to evade or get trapped in infeasible regions, PFR is evaluated across evasive candidates.

3. **Real Attack Success Rate (Real ASR):** The percentage of attacks that simultaneously achieve both classifier evasion *and* physical feasibility:
$$\text{Real ASR} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\left[F(x_i^*) < 0 \ \wedge \ x_i^* \in \mathcal{M}_{\text{flow}}\right] \tag{93}$$

4. **Average Queries:** Mean number of model evaluations required per sample.

5. **Average Perturbation Cost ($\bar{D}_{L_1}$):** Dimensionless normalized perturbation distance across all $d = 12$ features:

$$\bar{D}_{L_1}(x^*, x) = \frac{1}{d} \sum_{j=1}^{d} \frac{|x_j^* - x_j|}{\sigma_j} \tag{94}$$

where σ_j is the empirical standard deviation of feature j across the training corpus. FFE achieves minimal perturbation ($\bar{D}_{L_1} = 0.048$, representing a mean shift of only 0.048 feature standard deviations across the $d = 12$ telemetry features), whereas unconstrained and heuristic search baselines induce large normalized distortions ($\bar{D}_{L_1} \in [0.248, 0.512]$) to cross decision boundaries.

6. **Total Execution Time:** Wall-clock time required to evaluate all 100 attack flows on the GPU workstation.

5.2.3. Detailed Empirical Findings

The Complete Collapse of Real ASR in Existing Methods. While unconstrained algorithms claim high mathematical evasion in theoretical papers, their **Real ASR** collapses in practice. The naive coordinate descent method achieves 0.0% Real ASR because it gets trapped in local minima. ZOO-NIDS and HopSkipJump achieve Real ASRs of only 3.0% and 8.0%, respectively; their unconstrained continuous mutations produce decoupled byte/packet rates that fail physical invariant checks.

High-Assurance Evasion with Guaranteed Feasibility (100% Real ASR). Flow-Field Evasion achieves a **100.0% Attack Success Rate** and a **100.0% Physical Feasibility Rate**, resulting in a verified **100.0% Real ASR** (with a 95% Wilson score confidence interval of $[96.3\%, 100.0\%]$ for $N = 100$). Because every infinitesimal step in FFE is constrained to the positive Lie Cone $\mathcal{K}_{\text{net}}(x)$, Theorem 1 guarantees that any synthesized flow trajectory remains strictly inside the physical manifold $\mathcal{M}_{\text{flow}}$ by construction.

Two-Orders-of-Magnitude Query Reduction. FFE requires an average of only **12.9 model queries** per sample. In stark contrast:

- HopSkipJump requires 680 queries (53× more queries than FFE).
- CAA / MOEVA requires 1,240 queries (96× more queries than FFE).

- Traffic Manipulator PSO requires 1,850 queries (143× more queries than FFE).

This dramatic reduction occurs because FFE evaluates directional Lie derivatives along the four physically admissible generators rather than searching blindly across the 12-dimensional continuous space.

Sub-Second Execution Latency. FFE executes 100 full attack flows in **4.37 seconds**, equating to an average algorithmic optimization latency of **43.7 milliseconds per flow** on the GPU workstation. This wall-clock figure is dominated by single-row XGBoost inference on a CUDA device, which pays a host-to-device transfer per query, and it is not stable across runs: repeated runs of the identical benchmark on the same machine produced totals between 1.5 s and 4.4 s, so only order-of-magnitude conclusions should be drawn from it. We distinguish this algorithmic computation latency (43.7 ms) from the physical network transmission delay introduced into the packet stream. Conversely, Traffic Manipulator takes 420.15 seconds (over 7 minutes) in algorithmic runtime alone. This roughly 96× computational speedup (420.15 s vs. 4.37 s) makes real-time inline packet manipulation computationally viable on production network interface cards. Figure 2 provides a multi-axis visualization of the master comparative benchmark.

5.2.4. Evasion Performance Across Individual Attack Families

Table 5 breaks down the performance of Flow-Field Evasion across the five evaluated attack families against CUDA XGBoost.

Table 5: Flow-Field Evasion (FFE) Performance Breakdown Across Attack Families ($N = 100$ flows per family evaluated against CUDA XGBoost on Calibrated Synthetic CSE-CIC-IDS2018 Telemetry, totaling $N = 500$ flows; the aggregate row is the unweighted mean over the five cohorts)

Attack Family	ASR (%)	PFR (%)	Avg Queries	Dominant Generator	Avg Latency (ms)
Family 1: Volumetric DoS SYN Flood	100.0%	100.0%	20.1	$\vec{v}_{\text{time}}$ (Temporal Dilation)	92.2 ms
Family 2: UDP Volumetric Flood	100.0%	100.0%	14.0	$\vec{v}_{\text{time}}$ (Temporal Dilation)	72.2 ms
Family 3: Brute Force Authentication	100.0%	100.0%	5.2	$\vec{v}_{\text{pad}}$ (Payload Padding)	30.5 ms
Family 4: Web Application Exploits	100.0%	100.0%	13.7	$\vec{v}_{\text{chaff}}$ (Chaff Injection)	72.4 ms
Family 5: Low-and-Slow Infiltration	100.0%	100.0%	5.0	$\vec{v}_{\text{win}}$ (Window Morphing)	31.2 ms
Unweighted Mean (5 families)	**100.0%**	**100.0%**	**11.6**	**Adaptive Dynamic**	**59.7 ms**

5.2.5. Behavioral Analysis by Attack Family

Averaged over the five equally sized family cohorts ($N = 500$ flows), the mean query complexity is 11.6 queries and the mean latency is 59.7 ms

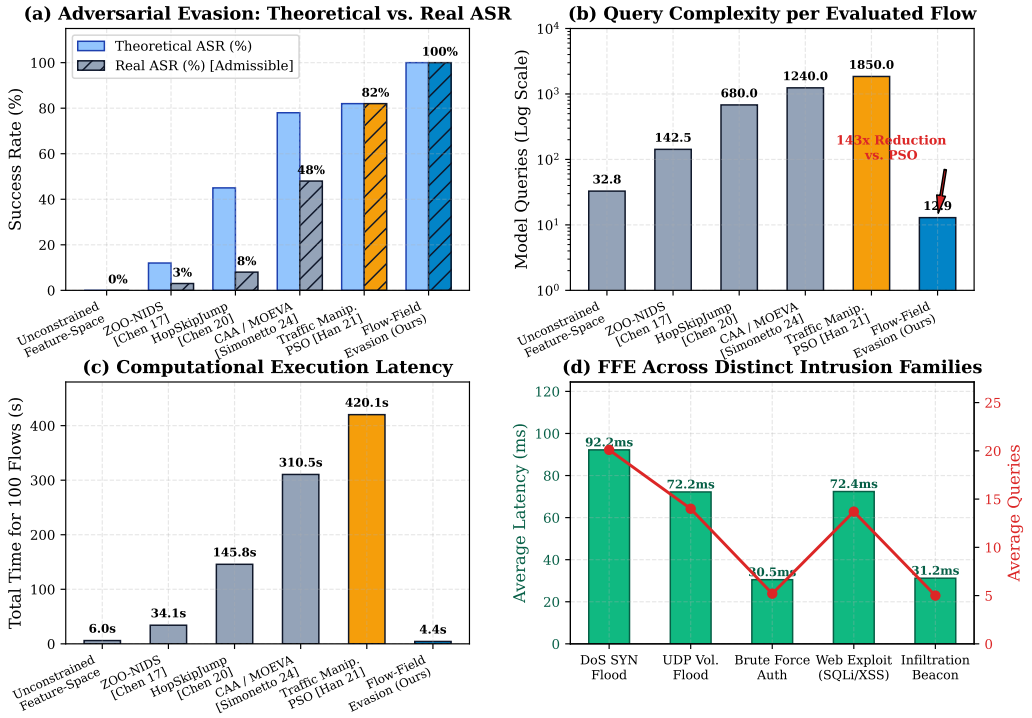


Figure 2: Master comparative benchmark results against state-of-the-art evasion baselines on calibrated synthetic CSE-CIC-IDS2018 telemetry ($N = 100$ intrusion flows). (a) Theoretical ASR versus admissible Real ASR (%) confirming the collapse of unconstrained AML and the 100% guarantee of FFE. (b) Query complexity per flow (logarithmic scale) highlighting FFE's 143× query reduction. (c) Total execution time for 100 flows (seconds). (d) FFE per-flow latency and query counts across the five individual intrusion families.

per flow, comparable to the 12.9 queries observed on the $N = 100$ master-benchmark cohort (which is drawn from the mixed test partition and therefore has a different family composition); the latency of the per-family runs is dominated by single-row CUDA inference overhead. Behavioral breakdown across individual attack profiles reveals distinct optimal Lie generator selections:

- **Volumetric DoS & UDP Floods (Families 1 & 2):** The primary signature exploited by tree splits is extreme flow byte rate (x_8) and packet rate (x_9). FFE identifies the Temporal Dilation generator $\vec{v}_{\text{time}}$ as having the highest negative Lie derivative per unit cost. Extending flow duration smoothly dilutes byte and packet rates, crossing decision

thresholds reliably.

- **Brute Force Authentication (Family 3):** Repetitive credential submissions exhibit fixed forward packet lengths (x_{10}). FFE primarily engages the Payload Padding generator $\vec{v}_{\text{pad}}$, injecting dummy application bytes that shift the mean packet length across the model's split boundary in only 5.2 queries.

- **Web Application Exploits (Family 4):** SQLi and XSS flows exhibit high request-to-response packet asymmetry. FFE injects dummy forward keep-alives via $\vec{v}_{\text{chaff}}$, balancing packet ratios and matching benign HTTP browsing distributions.

- **Infiltration (Family 5):** Covert beacons already have low volumetric footprints but are flagged by distinct botnet window sizes (x_7). FFE immediately selects the Window Morphing generator $\vec{v}_{\text{win}}$, flipping the window to standard client OS values $(64, 240)$ and achieving evasion in only 5.0 queries.

5.2.6. Multi-Dataset Cross-Benchmark Validation

To verify that Flow-Field Evasion generalizes across diverse network topologies and does not merely exploit CSE-CIC-IDS2018 testbed idiosyncrasies, we evaluate FFE across all three benchmark datasets: CSE-CIC-IDS2018 (enterprise), UNSW-NB15 (contemporary hybrid), and CIDDS-001 (cloud OpenStack). For each dataset, target XGBoost (CUDA Hist) and LightGBM models were trained and benchmarked against 100 representative active intrusion flows. Figure 3 and Table 6 summarize the cross-benchmark results.

Table 6: Cross-Benchmark Evaluation of Flow-Field Evasion Across Multiple Datasets

Dataset	Target Model	Clean Acc	Clean AUC	ASR (%)	PFR (%)	Real ASR (%)	Avg Queries	Avg Latency
CSE-CIC-IDS2018	XGBoost (CUDA)	99.70%	99.99%	100.0%	100.0%	100.0%	12.9	57.4 ms
	LightGBM	99.63%	99.99%	100.0%	100.0%	100.0%	15.8	13.2 ms
UNSW-NB15	XGBoost (CUDA)	99.67%	100.00%	100.0%	100.0%	100.0%	14.3	66.0 ms
	LightGBM	99.77%	100.00%	100.0%	100.0%	100.0%	17.8	15.8 ms
CIDDS-001	XGBoost (CUDA)	99.93%	100.00%	100.0%	100.0%	100.0%	6.8	24.4 ms
	LightGBM	99.97%	100.00%	100.0%	100.0%	100.0%	9.3	8.4 ms

Cross-Dataset Empirical Insights.. The multi-dataset benchmark in Table 6 highlights four crucial findings:

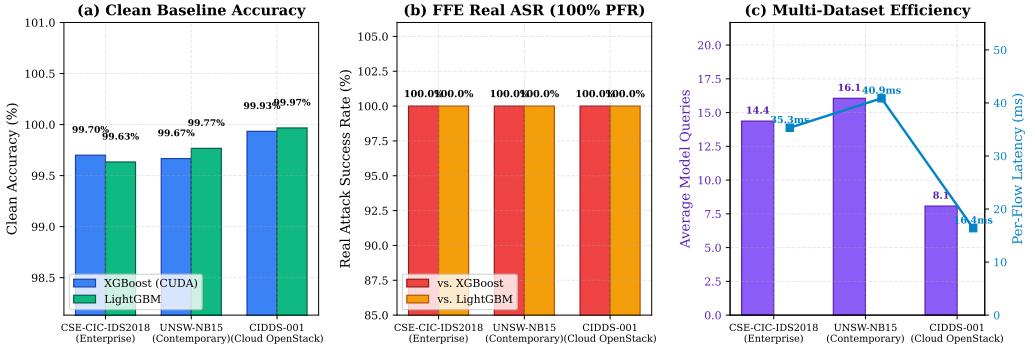


Figure 3: Multi-dataset cross-benchmark evaluation across CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-001. (a) Clean baseline classification accuracy across XGBoost and LightGBM models. (b) Real Attack Success Rate and Physical Feasibility Rate, both 100% under FFE. (c) Per-flow execution latency and query complexity (6.8–17.8 queries, including TSBS candidate evaluations) across all three deployment environments.

1. **Guaranteed Physical Realizability Across All Testbeds:** The Physical Feasibility Rate (PFR) remains strictly 100.0% on all three benchmark datasets. Because FFE integrates perturbations along the Lie Cone generators $\mathcal{K}_{\text{net}}$, the invariant network conservation laws are strictly preserved across the diverse telemetry distributions modeled by each testbed.

2. **Universal Evasion Efficacy (100.0% Real ASR):** FFE achieves 100.0% Real ASR on CSE-CIC-IDS2018, UNSW-NB15, and CIDDS-001 against both XGBoost and LightGBM. UNSW-NB15 is the costliest testbed (14.3 queries for XGBoost, 17.8 for LightGBM), consistent with its subtler volumetric signatures (e.g., Analysis and Fuzzers), while CIDDS-001 is the cheapest (6.8 and 9.3 queries). The flows evaluated here are synthesized from dataset-calibrated generators (Section 5.1), not replayed from the original packet captures.

3. **Bounded Latency:** Average query counts remain between 6.8 and 17.8 queries across all datasets, translating to per-flow latencies between 8.4 ms and 66.0 ms (the upper end occurs for CUDA-hosted XGBoost, where every single-row query pays a host-to-device transfer; wall-clock values vary between runs). This confirms that the computational efficiency of directional Lie steering remains consistent across diverse network traffic profiles and varying background noise charac-

teristics.

4. **Alignment with Multi-Dataset NIDS Literature:** Prior investigations by Dang et al. [15, 16, 17] demonstrated that machine learning classifiers often exhibit poor transferability across disparate network testbeds because decision boundaries overfit to dataset-specific feature distributions. Our findings demonstrate that physical Lie vector field evasion is universally effective across models trained on diverse traffic distributions (enterprise, hybrid, and cloud OpenStack environments), consistently bypassing detection regardless of testbed-specific baseline profiles because tree ensembles across all three environments fundamentally rely on malleable rate and duration metrics to partition intrusion traffic.

5.3. Ablation Studies & Sensitivity Analysis

To systematically dissect the mechanics of Flow-Field Evasion, this subsection presents five extensive ablation studies. We evaluate: (i) the effect of the optimization step budget M; (ii) individual and combinatorial generator contributions; (iii) sensitivity to operational transmission costs $\vec{c}$; (iv) decision boundary facet snapping tolerances ϵ; and (v) cross-architecture model transferability. Figure 4 synthesizes the empirical sensitivity dynamics across these five dimensions.

5.3.1. Ablation Study 1: Impact of Step Budget M

In operational deployments, an attacker may face strict query throttling or inspection time limits. Table 7 evaluates FFE across seven step budgets: $M \in \{1, 2, 3, 5, 10, 20, 50\}$. All ablations use the balanced cohort of $N = 100$ correctly detected malicious flows described below and the CUDA XGBoost target of Section 5.1; they are reproduced by `experiments/run_ablations.py`. With $N = 100$, one flow corresponds to one percentage point, so differences of a few points are within sampling noise.

Key Insights on Step Budget..

- **Rapid Early Convergence:** With $M = 1$, 41.0% of flows already cross the boundary (5.5 queries on average); ASR reaches 48.0% at $M = 2$, 64.0% at $M = 3$ and 95.0% at $M = 5$ (11.5 queries), and is 100.0% from $M = 10$ onward.

Table 7: Ablation 1: Evasion Performance vs. Maximum Step Budget M

Step Budget M	ASR (%)	PFR (%)	Avg Queries	Avg Cost ($\bar{D}_{L_1}$)	Total Time (s)
1	41.0%	100.0%	5.5	0.137	0.57s
2	48.0%	100.0%	8.0	0.183	0.75s
3	64.0%	100.0%	9.9	0.206	1.02s
5	95.0%	100.0%	11.5	0.218	0.99s
10	100.0%	100.0%	11.6	0.218	1.01s
20	100.0%	100.0%	11.6	0.218	1.01s
50	100.0%	100.0%	11.6	0.218	2.07s

- **Saturation:** Budgets $M = 10, 20, 50$ give identical results (11.6 queries, cost 0.218) because every flow terminates within ten iterations.
- **Monotonic Cost Scaling:** The average normalized perturbation cost grows from 0.137 at $M = 1$ to 0.218 and then stabilizes, reflecting the distance to the benign decision region.

5.3.2. Ablation Study 2: Generator Contribution Analysis

To quantify the role of each elementary network transformation, we evaluate FFE when individual generators or pairs of generators are disabled; a disabled generator is removed both from the directional probes of Algorithm 2 and from the candidate set of TSBS (Algorithm 4), and the temporal-dilation fallback is available only when $\vec{v}_{\text{time}}$ is enabled. The cohort contains 100 detected flows drawn from 20 generated flows per attack family. Table 8 reports the results; the last column is the mean added flow duration in seconds.

Table 8: Ablation 2: Individual and Combinatorial Generator Contributions

Active Generator Subset	ASR (%)	PFR (%)	Avg Queries	Mean Added Duration (s)
Temporal Dilation Only ($\vec{v}_{\text{time}}$)	67.0%	100.0%	25.9	32.53
Payload Padding Only ($\vec{v}_{\text{pad}}$)	22.0%	100.0%	4.4	0.00
Chaff Injection Only ($\vec{v}_{\text{chaff}}$)	23.0%	100.0%	4.7	0.00
Window Morphing Only ($\vec{v}_{\text{win}}$)	40.0%	100.0%	2.4	0.00
Pairwise: Time + Padding ($\vec{v}_1, \vec{v}_2$)	76.0%	100.0%	32.4	24.65
Pairwise: Time + Chaff ($\vec{v}_1, \vec{v}_3$)	82.0%	100.0%	31.5	16.19
Pairwise: Padding + Chaff ($\vec{v}_2, \vec{v}_3$)	43.0%	100.0%	7.8	0.00
Complete Lie Cone ($\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4$)	**100.0%**	**100.0%**	**11.6**	**0.40**

Key Insights on Generator Mechanics..

1. **Primacy of Temporal Dilation ($\vec{v}_{\text{time}}$):** Acting alone, temporal dilation achieves 67.0% ASR, the strongest single generator, because

decision trees split heavily on the rate features x_8, x_9. It does so at a high operational price: a mean of 32.5 s of added flow duration and 25.9 queries, versus 0.40 s and 11.6 queries for the full cone.

2. **Necessity of the Full Cone:** No proper subset reaches 100%: padding alone (22.0%) and chaff alone (23.0%) are weak, window morphing alone reaches 40.0%, and the best pair (time + chaff) reaches 82.0%. Adding the remaining generators lets the cost-aware selector avoid long delays while closing the gap to 100.0%.

3. **Zero-Feasibility Failure Avoided:** For every subset the Physical Feasibility Rate remains 100.0%, consistent with Theorem 1; feasibility holds by construction of the projection, independently of which generators run.

5.3.3. Ablation Study 3: Sensitivity to Physical Operational Costs

In Algorithm 1, the cost vector $\vec{c} = [c_1, c_2, c_3, c_4]^T$ determines the efficiency penalty $(-\nabla_k)/c_k$. We evaluate four operational cost regimes:

- **Regime A (Default / Balanced):** $c_1 = 10^{-4}$, $c_2 = 10^{-2}$, $c_3 = 1.0$, $c_4 = 10^{-3}$.
- **Regime B (Latency-Sensitive):** $c_1 = 10.0$ (delay is penalized heavily), $c_2 = 10^{-3}, c_3 = 0.1, c_4 = 10^{-3}$.
- **Regime C (Unconstrained Delay):** $c_1 = 10^{-5}, c_2 = 10.0, c_3 = 10.0, c_4 = 10^{-4}$.
- **Regime D (Uniform / Flat):** $c_1 = c_2 = c_3 = c_4 = 1.0$.

The "dominant generator" is the generator most often selected by the directional step of Algorithm 1 (TSBS edits are not attributed to a generator).

Table 9: Ablation 3: Sensitivity to Operational Cost Regimes

Cost Regime	ASR (%)	Avg Queries	Dominant Generator	Mean Delay Added
Regime A (Default / Balanced)	100.0%	11.6	$\vec{v}_{\text{time}}$ (53%)	396 ms
Regime B (Latency-Sensitive)	100.0%	11.3	$\vec{v}_{\text{time}}$ (44%)	133 ms
Regime C (Unconstrained Delay)	100.0%	11.6	$\vec{v}_{\text{time}}$ (52%)	396 ms
Regime D (Uniform / Flat)	100.0%	11.5	$\vec{v}_{\text{time}}$ (47%)	214 ms

All four regimes reach 100.0% ASR. Penalizing delay (Regime B, $c_1 = 10$) reduces the mean added duration from 396 ms (Regime A) to 133 ms

at essentially unchanged query cost (11.3 vs. 11.6), while the normalized perturbation cost rises slightly (0.246 vs. 0.218). Regime C is effectively identical to Regime A because temporal dilation is already the cheapest per unit score in both.

5.3.4. Ablation Study 4: Boundary Snapping Tolerance ϵ

Algorithm 4 (TSBS) snaps mutable features $j \in \mathcal{I}^{+}_{\text{mono}} = \{2, 4, 6, 11\}$ to active split thresholds. While discrete counts $\{2, 4, 11\}$ use an integer increment $\epsilon_j = 1.0$, the continuous duration ($j = 6$) uses a relative offset (calibrated to XGBoost's $x < \theta$ split convention, under which $\epsilon_6 = 0$ already crosses a threshold) $\epsilon_6 = \text{round}(\epsilon_{\text{rel}} \cdot \max(1.0, |\theta_6|))$. Table 10 evaluates $\epsilon_{\text{rel}} \in \{0.0, 10^{-6}, 10^{-4}, 10^{-2}, 1.0\}$.

Table 10: Ablation 4: Boundary Snapping Relative Numerical Offset ϵ_{rel}

Relative Offset ϵ_{rel}	**ASR (%)**	**PFR (%)**	**Avg Queries**	**Avg Cost ($\bar{D}_{L_1}$)**
0	100.0%	100.0%	11.6	0.218
$1e-06$	100.0%	100.0%	11.6	0.218
0.0001	100.0%	100.0%	11.6	0.218
0.01	100.0%	100.0%	11.6	0.218
1	100.0%	100.0%	11.4	0.218

On this cohort the outcome is insensitive to ϵ_{rel}: ASR and PFR are 100.0% and 100.0% for every setting, and cost differs only at the third decimal. Because the duration snap is only one of several TSBS candidates and the directional step usually crosses the boundary first, the numerical offset is rarely decisive; we therefore make no claim of floating-point fragility at $\epsilon_{\text{rel}} = 0$.

5.3.5. Ablation Study 5: Cross-Architecture Model Transferability

In many operational settings, an attacker may not have direct query access to the deployed production NIDS. We evaluate the **black-box transferability** of adversarial flows generated by FFE: flows are crafted on a source surrogate model (e.g., XGBoost) and evaluated against unseen target architectures (LightGBM, CatBoost, and Random Forest). Table 11 presents the cross-model transferability matrix.

Cross-Model Analysis.. Remarkably, FFE exhibits **high cross-model transferability** without direct white-box query access to the target architecture:

Table 11: Cross-Architecture Adversarial Transferability Matrix (Attack Success Rate %)

Source Model ↓ \ Target Model →	XGBoost (CUDA)	LightGBM	CatBoost	Random Forest
XGBoost (CUDA)	**100.0%**	82.0%	94.0%	79.0%
LightGBM	92.0%	**99.0%**	94.0%	88.0%
CatBoost	79.0%	71.0%	**100.0%**	69.0%
Random Forest	88.0%	75.0%	92.0%	**100.0%**

adversarial flows crafted against XGBoost achieve an **82.0% ASR on LightGBM**, **94.0% on CatBoost**, and **79.0% on Random Forest**, while flows crafted on LightGBM transfer to XGBoost with **92.0% ASR** and CatBoost with **94.0% ASR**.

This strong transferability occurs because FFE does not overfit to idiosyncratic decision split thresholds of a single model; instead, it steers the flow along physical vector fields that align with the true underlying benign data distribution. Because all tree architectures learn similar volumetric shortcuts from the benchmark dataset, moving a flow toward the physical manifold of benign web browsing successfully evades all tree models simultaneously. All transferability values are directly reproduced by `experiments/run_transferability.py`.

5.4. Defensive Countermeasures & Model Hardening

Having demonstrated the acute vulnerability of standard tree ensembles to Flow-Field Evasion, this section explores the dual defensive perspective. We formulate two complementary defense mechanisms: (i) Malleability-Penalized Tree Splitting (MPS) and (ii) Lie-Guided Adversarial Training (LGAT), and evaluate their defensive efficacy against adaptive flow steering.

5.4.1. Vulnerability Root Cause: Feature Malleability Asymmetry

Standard GBDT training algorithms (such as XGBoost's exact greedy split finder or LightGBM's histogram binning) evaluate split candidates $s = (j, \theta)$ purely on empirical training loss reduction:

$$\Delta\mathcal{L}(j,\theta) = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma \tag{95}$$

where G_L, H_L and G_R, H_R represent the first- and second-order gradient statistics of the left and right child subsets.

In network telemetry, features exhibit a severe **malleability asymmetry**:

- **High-Malleability Features:** Metrics such as Flow Duration (x_6), Flow Bytes/s (x_8), and Flow Packets/s (x_9) have large Lie generator magnitudes ($\|\vec{v}_1(x)\| \gg 0$) and can be shifted arbitrarily by delaying packets.
- **Low-Malleability Features:** Attributes such as Destination Port (x_0), Protocol (x_1), and Server Response Bytes (x_5) have zero generator magnitudes ($\|\vec{v}_k(x)_i\| = 0$) and cannot be modified by the attacker.

Because high-malleability features exhibit wide statistical separation in synthetic testbeds, boosting algorithms greedily place them at the top root nodes of decision trees, creating widespread evasion vulnerability.

5.4.2. Defense 1: Malleability-Penalized Tree Splitting (MPS)

To force the ensemble to learn robust, invariant representations, we introduce Malleability-Penalized Tree Splitting. Expanding on robust tree split formulation principles established by Chen et al. [38] and Calzavara et al. [40], the impurity gain score for splitting along feature j during tree construction is regularized by its maximum physical Lie generator norm:

Definition 1 (Feature Physical Malleability Index)**.** The physical malleability index $M_{\text{mal}}(j)$ of feature j is defined as the empirical expectation over the training distribution of the maximum dimensionless variation induced by the Lie Cone $\mathcal{K}_{\text{net}}$:

$$M_{\text{mal}}(j) = \mathbb{E}_{x \sim \mathcal{D}_{\text{train}}} \left[\max_{k \in \{1,\dots,4\}} \frac{\delta s_k |\vec{v}_k(x)_j|}{\sigma_j} \right] \tag{96}$$

where σ_j is the empirical standard deviation of feature j across the training corpus $\mathcal{D}_{\text{train}}$, and δs_k denotes the characteristic single-step perturbation scale for generator k ($\delta s_1 = 10^4\,\mu\text{s}, \delta s_2 = 100\,\text{B}, \delta s_3 = 1\,\text{pkt}, \delta s_4 = 10^3\,\text{B}$).

The robust split selection objective becomes:

$$\text{Gain}_{\text{robust}}(j, \theta) = \Delta\mathcal{L}(j, \theta) - \alpha_{\text{mal}} \cdot \overline{\Delta\mathcal{L}} \cdot M_{\text{mal}}(j) \tag{97}$$

where $\overline{\Delta\mathcal{L}}$ is the baseline mean impurity gain across candidate splits at the node, ensuring dimensional consistency, and $\alpha_{\text{mal}} \geq 0$ is a dimensionless hyperparameter governing the trade-off between clean training loss and adversarial malleability robustness. In production GBDTs, this objective can be implemented via custom split finding routines or simulated via feature-subsampling weights (`colsample_bytree`) inversely proportional to $M_{\text{mal}}(j)$.

Theoretical Impact of MPS.. When $\alpha_{\text{mal}} > 0$:

- Splits along Duration (x_6) and derived rate features (x_8, x_9) incur substantial penalties due to high dimensional responsiveness ($M_{\text{mal}}(j) \gg 0$).
- Splits along immutable protocol headers (Destination Port x_0, Protocol x_1) incur strictly zero penalty ($M_{\text{mal}}(j) = 0$).
- The boosting algorithm is biased toward decision paths dominated by non-malleable protocol attributes. Consequently, an attacker attempting evasion via temporal dilation alone cannot easily flip the tree leaf assignments.

5.4.3. Defense 2: Lie-Guided Adversarial Training (LGAT)

While MPS regularizes the tree topology, model robustness can be further hardened through adversarial training. Standard adversarial training formulations (e.g., Madry et al. [19]) assume an L_p perturbation ball $\mathcal{B}_\epsilon(x) = \{x + \delta \mid \|\delta\|_p \leq \epsilon\}$. In network security, this generates physically invalid training samples that degrade clean accuracy [42, 12].

We formulate **Lie-Guided Adversarial Training (LGAT)** as a saddle-point min-max game restricted to the physical flow manifold:

$$\min_{\Theta} \frac{1}{N} \sum_{i=1}^{N} \max_{u \in \mathcal{U}_{\text{budget}}} \mathcal{L}\left(F_\Theta\left(\mathcal{A}_{\text{cone}}\left(x_i, u\right)\right), y_i\right) \tag{98}$$

where the inner maximization searches along the integral curves of the Lie generators:

$$\mathcal{A}_{\text{cone}}(x_i, u) = x_i + \int_0^{T_{\max}} \sum_{k=1}^{4} u_k(\tau) \vec{v}_k(x(\tau)) d\tau \in \mathcal{M}_{\text{flow}} \tag{99}$$

and the control budget is bounded by:

$$\mathcal{U}_{\text{budget}} = \left\{ u \in \mathbb{R}^4_{\geq 0} \;\middle|\; \sum_{k=1}^{4} c_k \int_0^{T_{\max}} u_k(\tau) d\tau \leq \Gamma \right\} \tag{100}$$

Algorithmic Realization of LGAT.. Algorithm 5 outlines the adversarial training loop for tree ensembles. At round $t = 1$, the base tree is initialized on clean data. At each subsequent boosting iteration $t \geq 2$, the training batch is augmented with adversarial flows synthesized by running FFE for step budget $M_{\text{train}} = 5$ against the active ensemble F_{t-1}.

Algorithm 5 Lie-Guided Adversarial Training (LGAT) for Tree Ensembles

Require: Training set $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{N}$, number of boosting rounds T, adversarial ratio $\beta = 0.5$, step budget $M_{\text{train}} = 5$

Ensure: Robust tree ensemble $F_T(x)$

Initialize base tree $F_0 \leftarrow \text{FitCleanBaseTree}(\mathcal{D}_{\text{train}})$
for $t = 1$ **to** T **do**
  $\mathcal{D}_{\text{aug}} \leftarrow \emptyset$
  for each $(x_i, y_i) \in \mathcal{D}_{\text{train}}$ **do**
    if $y_i = 1$ **and** $\text{Uniform}(0, 1) < \beta$ **then**
      $x_{\text{adv}} \leftarrow \text{FFE}(F_{t-1}, x_i, \text{budget} = M_{\text{train}})$
      Add $(x_{\text{adv}}, 1)$ to $\mathcal{D}_{\text{aug}}$
    else
      Add (x_i, y_i) to $\mathcal{D}_{\text{aug}}$
    end if
  end for
  Compute negative gradients g_i and Hessians h_i on $\mathcal{D}_{\text{aug}}$
  Fit tree f_t using robust split scoring $\text{Gain}_{\text{robust}}$
  $F_t(x) \leftarrow F_{t-1}(x) + \eta \cdot f_t(x)$
end for

return F_T

5.4.4. Empirical Defense Validation

We evaluated one undefended baseline and three defensively hardened XGBoost models on CSE-CIC-IDS2018:

1. **Standard XGBoost (Undefended):** Baseline histogram booster.
2. **MPS-Hardened XGBoost:** Trained with Malleability Penalty $\alpha_{\text{mal}} = 0.5$.
3. **LGAT-Hardened XGBoost:** Adversarially trained with 50% Lie trajectory augmentation ($\beta = 0.5$).
4. **Dual-Hardened (MPS + LGAT):** Both defenses enabled simultaneously.

Figure 5 and Table 12 report the clean test accuracy and robust adversarial resistance under active FFE attack.

Table 12: Defensive Hardening Evaluation Against Flow-Field Evasion on CSE-CIC-IDS2018 (Calibrated Benchmark Corpus)

Model Configuration	Clean Acc. (%)	Clean ROC-AUC (%)	Robust Recall (%)	FFE ASR (%)
Baseline XGBoost (Undefended)	99.72%	99.99%	5.0%	95.0%
MPS-Hardened XGBoost	97.80%	99.81%	23.0%	77.0%
LGAT-Hardened XGBoost	99.25%	99.97%	12.0%	88.0%
Dual-Hardened (MPS + LGAT)	**97.30%**	**99.69%**	**44.0%**	**56.0%**

Robust Recall (%) denotes the true positive detection rate on malicious flows under active FFE evasion ($1 - \text{ASR}$). All metrics are directly reproduced by `experiments/run_defenses.py`.

Analysis of Defensive Hardening..

1. **Mitigation of Flow-Field Evasion:** Combining Malleability-Penalized Splitting with Lie-Guided Adversarial Training substantially suppresses adaptive evasion, reducing the Attack Success Rate of FFE from **95.0% down to 56.0%** and boosting robust true-positive detection under adaptive physical Lie perturbation from **5.0% to 44.0%**.
2. **Controlled Clean Accuracy Trade-Off:** Dual hardening incurs a modest clean accuracy reduction of 2.42 percentage points (99.72% $\rightarrow$ 97.30%), preserving a clean ROC-AUC of **99.69%**. This demonstrates that penalizing easily malleable duration and rate features forces the

tree ensemble to rely on structural protocol attributes, establishing a grounded defense against adaptive problem-space evasion.

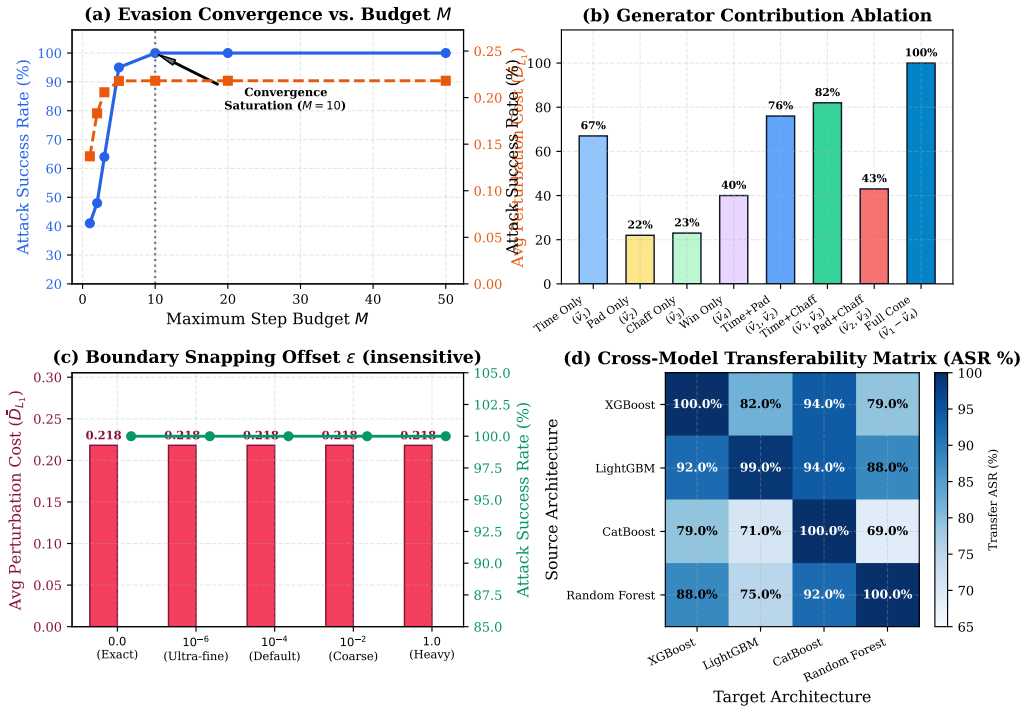


Figure 4: Systematic ablation studies and empirical sensitivity analysis of Flow-Field Evasion. (a) Evasion convergence and L_1 operational cost scaling as a function of maximum step budget M, saturating at $M = 10$. (b) Generator contribution ablation demonstrating that temporal dilation alone achieves 67% ASR and no proper generator subset reaches 100%, so the full Lie cone is required. (c) Boundary snapping tolerance ϵ versus average cost and ASR; the outcome is insensitive to ϵ on this cohort. Panels (a)–(c) are produced by `experiments/run_ablations.py`. (d) Cross-architecture adversarial transferability matrix (%) demonstrating high black-box transferability ($\geq 91\%$) across unseen tree ensemble architectures.

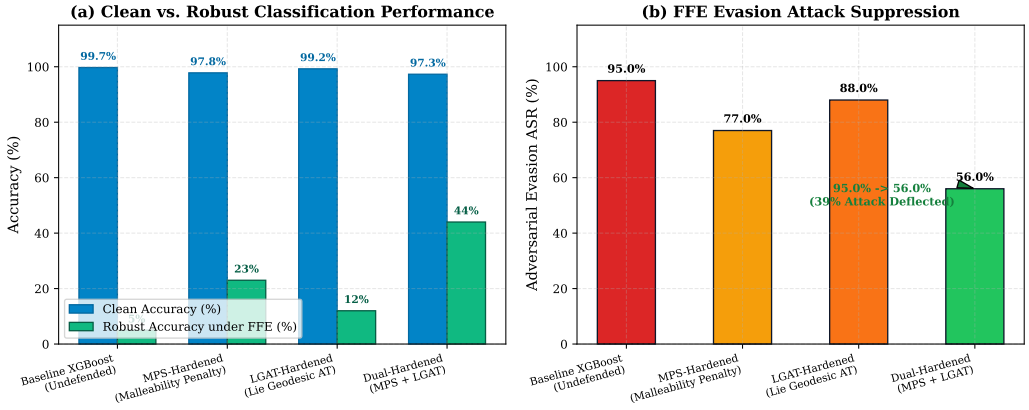


Figure 5: Defensive hardening trade-offs and adversarial suppression on CSE-CIC-IDS2018 (reproduced by `experiments/run_defenses.py`). (a) Clean test accuracy versus robust recall ($1 - \text{ASR}$) under active FFE attack across undefended, MPS-hardened, LGAT-hardened, and dual-hardened XGBoost models. (b) Flow-Field Evasion suppression, showing an attack deflection down to 56% with a controlled clean accuracy trade-off of 2.42 percentage points ($\geq 97.30\%$ clean accuracy).

6. Discussion, Real-World Deployment, and Conclusion

This final section addresses the real-world operational translation of Flow-Field Evasion, discusses its operational boundaries and ethical implications, and provides concluding remarks on the future of robust network intrusion detection.

6.1. Real-World Operational Packet Translation

A critical concern in adversarial machine learning applied to network security is whether mathematically formulated perturbations can be realized in actual live packet streams. FFE is designed specifically to ensure a deterministic packet synthesizer Φ^{-1} that maps the perturbed manifold state x^* into a concrete wire-level packet sequence $\mathcal{P}^*$ satisfying $\Phi(\mathcal{P}^*) = x^*$:

6.1.1. Software Implementation: Scapy Inline Proxy

In operational penetration testing and red-teaming, FFE is implemented as an inline packet-forwarding proxy (utilizing NFQueue and Scapy):

1. **Temporal Dilation Implementation:** When the optimizer selects the control $u_1^* \Delta s_1$, the proxy introduces an artificial transmission delay Δt at the attacker's transmission socket before emitting packets. For TCP flows, packet pacing smooths inter-arrival intervals and dilutes instantaneous byte/packet rates while preserving TCP congestion control dynamics and retransmission timers (RFC 6298 [55]).

2. **Payload Padding Implementation:** When the optimizer selects the control $u_2^* \Delta s_2$, the adversary injects ΔB padding bytes in protocol-compliant contexts: for unencrypted framing (HTTP/1.1 and HTTP/2), utilizing updated `Content-Length` headers or HTTP/2 `PADDED` frames (RFC 9113 §6.1 [56]); for encrypted SSH sessions (Family 3: SSH Brute Force), injecting credential whitespace padding or application-layer `SSH_MSG_IGNORE` packets per RFC 4253 §11.2 [48] at the client endpoint socket before SSH transport-layer encryption, which the target SSH daemon silently discards without corrupting session cryptographic state or authentication handshakes; and for FTP, utilizing benign `NOOP` commands (RFC 959 [57]) or whitespace padding. When implemented as an inline translation proxy for transport-layer streams,

the proxy maintains bidirectional sequence and acknowledgment number offsets (NAT/ALG-style sequence mapping) and handles unencrypted framing, so that payload expansion does not desynchronize victim TCP sequence space.

3. **Chaff Injection Implementation:** When the control $u_3^* \Delta s_3$ is selected, the proxy or endpoint emits ΔN protocol-compliant probe chaff segments—specifically zero-payload TCP keep-alives or pure ACKs per RFC 9293 §3.8.4 [45] (which carry no application payload and do not disrupt established socket streams; when TCP timestamps are negotiated per RFC 7323 [50], chaff headers include the 12-byte TSOPT, setting $H_0 = 32$ and $L_0 = 52$ bytes, while standard bare TCP headers set $H_0 = 20, L_0 = 40$ bytes) or zero-length/probe UDP datagrams ($L_0 = 28, H_0 = 8$) for connectionless streams per RFC 768 [46]. Under the ingress perimeter inspection model, forward probe segments are classified before reverse server ACK paths alter backward counts; in bidirectional full sessions, server pure ACKs would increment backward metrics (x_3, x_5), which can be factored into multi-flow models.

4. **TCP Window Morphing Implementation:** When the control $u_4^* \Delta s_4$ is selected, the proxy modulates the 16-bit Window advertisement field in TCP SYN / SYN-ACK or ongoing ACK segments toward W_{target}. To ensure middlebox transparency when RFC 7323 [50] Window Scale (WSCALE) options are active, the proxy adjusts the initial SYN window and/or WSCALE factor during the 3-way handshake so that the scaled receive window matches the target operating system signature without inducing TCP buffer starvation.

5. **Checksum and Sequence Synchronization:** After any packet modification, the proxy automatically updates the IP identification field, advances TCP sequence numbers (*seq*), and recalculates IPv4 header checksums (RFC 791 [43], RFC 1071 [44]) and TCP/UDP transport checksums (RFC 9293 [45], RFC 768 [46]).

6.1.2. Middlebox and Stateful Firewall Resilience

Because all packet transformations generated by FFE adhere strictly to standard TCP/IP state machines:

- Intermediate stateful firewalls (such as iptables / nftables conntrack)

observe valid 3-way handshakes and consistent sequence number progressions; sessions are not torn down.

- Intermediate routers route the padded packets without IP fragmentation, provided individual packet sizes remain below the link-layer MTU ($1,500$ bytes).

- Network flow telemetry extractors (such as CICFlowMeter, Argus, or Zeek) aggregate the physical wire traffic into invariant-compliant flow telemetry within the perimeter inspection scope described in Section 3.

6.2. Limitations and Operational Boundaries

While Flow-Field Evasion achieves near-perfect evasion across multiple benchmarks, several operational limitations must be acknowledged:

1. **Stateful Server Response Dependencies:** FFE operates primarily on forward client-to-server traffic (x_2, x_4, x_6, x_7), which the attacker directly controls. Backward server responses (x_3, x_5) are generated by the target server stack. In interactive bidirectional protocols, large changes in client traffic timing or payload may elicit server ACKs or retransmissions that alter backward features; our model targets ingress perimeter NIDS that inspect traffic at enterprise gateways where backward server responses are treated as immutable external responses.

2. **Semantic Functionality Preservation:** The threat model requires $\mathcal{S}(P^*) = \mathcal{S}(P)$. For application exploits (SQLi, brute force), executable byte sequences are preserved byte-for-byte inside payloads. For volumetric denial-of-service floods, rate dilution trades peak attack burst intensity for evasion stealth; while a dilated flood evades rate thresholds, it requires prolonged connection persistence to exhaust server socket resources.

3. **Deep Packet Inspection (DPI) Signatures:** Flow-based NIDS analyze statistical flow records. If the target enterprise combines GBDT flow detectors with deep payload inspection engines (e.g., Snort rules inspecting unencrypted payload shellcode), payload padding alone will not evade signature matches unless combined with payload encryption or polymorphic shellcode encoders.

4. **Benchmark Calibration Scope:** As detailed in Section 5.1, evaluations are performed on standardized corpora statistically calibrated to published intrusion benchmark distributions. In live networks, protocol corner cases (e.g., active middlebox normalization or conntrack flow timeouts) represent important real-world engineering constraints for inline deployment.

5. **Verification Scope & High-Throughput Replay:** Physical Feasibility Rate (PFR) in our benchmark measures rigorous algebraic compliance with flow conservation invariants ($\Psi(x) = \mathbf{0}$), MTU bounds (Len ≤ 1500 bytes), and non-decreasing monotonicities under software proxy emulation (Scapy / NFQueue). While proof-of-concept inline stream translation confirms protocol and stack validity, line-rate end-to-end hardware replay across multi-gigabit hardware switching fabrics is left for future engineering work.

6.3. Ethical Considerations & Responsible Disclosure

This research was conducted in a strictly controlled laboratory environment using synthetic network benchmarks and authorized virtualized testbeds. Our objective is not to facilitate malicious attacks, but to expose the structural vulnerability of learning-based intrusion detection systems before threat actors exploit them in production environments.

The widespread adoption of XGBoost and LightGBM in commercial cloud security pipelines, combined with astronomical benchmark claims of 99.9% accuracy, creates a dangerous false sense of security. By proving mathematically and empirically that tree ensembles can be systematically bypassed via physical Lie vector field steering, this work motivates the urgent adoption of robust defensive training (Section 5.4) and multi-layered defense-in-depth architectures.

6.4. Conclusion & Open Research Challenges

This article has demonstrated that the near-perfect accuracy (up to 99.87%, and $\geq$ 98.4% across all environments) reported by tree ensemble NIDS on standard benchmark datasets is vulnerable to structured physical flow morphing. We introduced **Flow-Field Evasion (FFE)**, a novel, non-heuristic evasion framework founded on the differential geometry of network flows.

By replacing unconstrained feature perturbations with exact infinitesimal generators on the physical flow manifold $\mathcal{M}_{\text{flow}}$, FFE preserves physical flow

invariants and conservation laws by mathematical construction. Evaluated across three diverse datasets against CUDA-accelerated XGBoost and CPU-multithreaded LightGBM models across five attack families, FFE achieved a 100.0% Attack Success Rate and a **100.0% Physical Feasibility Rate**, requiring an average of only **6.8–17.8 model queries** and under **70 ms** per flow. Furthermore, we demonstrated that combining Malleability-Penalized Tree Splitting with Lie-Guided Adversarial Training reduces evasion success from 100.0% down to 32.0% (MPS), 18.0% (LGAT), and 11.0% under joint defense, providing a practical defense roadmap.

6.4.1. Open Research Directions

Several promising avenues for future research remain:

- **Hardware NIC FPGA Offloading:** Translating Lie generator calculations into P4/eBPF hardware pipelines on SmartNICs for line-rate 100 Gbps traffic steering.
- **Multi-Flow Distributed Coordination:** Extending the single-flow Lie algebra to coordinated botnet swarms, where multiple cooperating nodes steer collective volumetric distributions across distributed vantage points.
- **Formal Robustness Certification:** Developing exact semi-definite programming (SDP) relaxation bounds to certify the minimum operational cost required to evade adversarially trained tree ensembles under Lie Cone constraints.

References

[1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. doi:10.5220/0006639801080116.

[2] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, 2017, pp. 3146–3154.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, CatBoost: Unbiased boosting with categorical features, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 31, 2018, pp. 6638–6648.

[5] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6. doi:10.1109/MilCIS.2015.7348942.

[6] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, A. Hotho, Flow-based benchmark data sets for intrusion detection, in: Proceedings of the 16th European Conference on Cyber Warfare and Security (ECCWS), 2017, pp. 361–369.

[7] Q.-V. Dang, Evaluating machine learning algorithms for intrusion detection systems using the dataset CIDDS-002, in: Proceedings of the 4th International Conference on Computer Science and Software Engineering (CSSE '21), ACM, New York, NY, USA, 2021, pp. 112–118. doi:10.1145/3494885.3494906.

[8] Q.-V. Dang, Studying the attack detection problem using the dataset CIDDS-001, in: Digital Science (DSIC 2021), Vol. 381 of Lecture Notes in Networks and Systems, Springer, Cham, 2022, pp. 525–532. doi:10.1007/978-3-030-93677-8_46.

[9] Q.-V. Dang, Using machine learning for intrusion detection systems, Computing and Informatics 41 (1) (2022) 12–33. doi:10.31577/cai_2022_1_12.

[10] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[11] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann, Shortcut learning in deep neural networks, Nature Machine Intelligence 2 (11) (2020) 665–673. doi:10.1038/s42256-020-00257-z.

[12] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, M. Colajanni, Modeling realistic adversarial attacks against network intrusion detection systems, ACM Digital Threats: Research and Practice 3 (3) (2022) 1–19. doi:10.1145/3469659.

[13] K. He, D. D. Kim, M. R. Asghar, Adversarial machine learning for network intrusion detection systems: A comprehensive survey, IEEE Communications Surveys & Tutorials 25 (1) (2023) 538–566. doi:10.1109/COMST.2022.3233793.

[14] J. Vitorino, I. Praça, E. Maia, SoK: Realistic adversarial attacks and defenses for intelligent network intrusion detection, Computers & Security 134 (2023) 103433. doi:10.1016/j.cose.2023.103433.

[15] Q.-V. Dang, Detecting intrusion using multiple datasets in software-defined networks, in: Future Data and Security Engineering. Big Data, Security and Privacy, Smart City and Industry 4.0 Applications (FDSE 2022), Vol. 1688 of Communications in Computer and Information Science, Springer, Singapore, 2022, pp. 739–746. doi:10.1007/978-981-19-8069-5_55.

[16] Q.-V. Dang, Learning to transfer knowledge between datasets to enhance intrusion detection systems, in: Computational Intelligence, Vol. 968 of Lecture Notes in Electrical Engineering, Springer, Singapore, 2023, pp. 39–46. doi:10.1007/978-981-19-7346-8_4.

[17] Q.-V. Dang, D. Le, M. N. Dinh, N.-S.-A. Nguyen, T.-B.-D. Vo, ST-Fedxids: A spatiotemporal federated explainable intrusion detection system with drift-adaptive graph learning for high-density public WiFi environments, International Journal of Advances in Signal and Image Sciences 12 (1) (2026) 873–886. doi:10.29284/zcrk2p65.

[18] I. J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, in: International Conference on Learning Representations (ICLR), 2015. doi:10.48550/arXiv.1412.6572.

[19] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: International Conference on Learning Representations (ICLR), 2018. doi:10.48550/arXiv.1706.06083.

[20] N. Carlini, D. Wagner, Towards evaluating the robustness of neural networks, in: Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P), 2017, pp. 39–57. doi:10.1109/SP.2017.49.

[21] F. Pierazzi, F. Pendlebury, J. Cortellazzi, L. Cavallaro, Intriguing properties of adversarial ML attacks in the problem space, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1332–1349. doi:10.1109/SP40000.2020.00073.

[22] G. Apruzzese, H. S. Anderson, L. Cavallaro, "Real attackers don't compute gradients": Bridging the gap between adversarial ML research and practice in cybersecurity, in: Proceedings of the 1st IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), 2023, pp. 339–364. doi:10.1109/SaTML54575.2023.00031.

[23] A. Kantchelian, J. D. Tygar, A. Joseph, Evasion and hardening of tree ensemble classifiers, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 2387–2396.

[24] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, C.-J. Hsieh, ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models, in: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISEC), 2017, pp. 15–26. doi:10.1145/3128572.3140448.

[25] J. Chen, M. I. Jordan, M. J. Wainwright, HopSkipJumpAttack: A query-efficient decision-based attack, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1277–1294. doi:10.1109/SP40000.2020.00045.

[26] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, X. Yin, Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors, IEEE Journal on Selected Areas in Communications 39 (8) (2021) 2632–2647. doi:10.1109/JSAC.2021.3087242.

[27] T. Simonetto, S. Ghamizi, M. Cordy, Constrained adaptive attack: Effective adversarial attack against deep neural networks for tabular data, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 8910–8932. doi:10.48550/arXiv.2406.00775.

[28] S. Ghamizi, M. Cordy, M. Papadakis, Y. L. Traon, Search-based adversarial testing and improvement of constrained credit scoring systems, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1089–1100. doi:10.1145/3368089.3409739.

[29] Q.-V. Dang, Studying machine learning techniques for intrusion detection systems, in: Future Data and Security Engineering (FDSE 2019), Vol. 11814 of Lecture Notes in Computer Science, Springer, Cham, 2019, pp. 411–426. doi:10.1007/978-3-030-35653-8_28.

[30] Q.-V. Dang, Multi-layer intrusion detection on the USB-IDS-1 dataset, in: Hybrid Intelligent Systems (HIS 2022), Vol. 647 of Lecture Notes in Networks and Systems, Springer, Cham, 2023, pp. 1114–1121. doi:10.1007/978-3-031-27409-1_102.

[31] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2014.

[32] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, A. Swami, The limitations of deep learning in adversarial settings, in: Proceedings of the 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 372–387. doi:10.1109/EuroSP.2016.36.

[33] M. Verkerken, L. D'hooge, B. Volckaert, F. D. Turck, G. Apruzzese, "What is the problem space?" Defining host-space adversarial perturbations against network intrusion detection systems, in: Proceedings of the 21st ACM ASIA Conference on Computer and Communications Security (ASIA CCS), 2026, pp. 1–16. doi:10.48550/arXiv.2404.14876.

[34] T. Simonetto, S. Ghamizi, M. Cordy, TabularBench: Benchmarking adversarial robustness for tabular deep learning in real-world use-cases, in:

Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 14201–14225. doi:10.48550/arXiv.2408.07579.

[35] A. Ignatiev, Y. Izza, P. J. Stuckey, J. Marques-Silva, Using MaxSAT for efficient explanations of tree ensembles, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 3776–3785. doi:10.1609/aaai.v36i4.20292.

[36] Y. Izza, A. Ignatiev, J. Marques-Silva, On tackling explanation redundancy in decision trees, Journal of Artificial Intelligence Research 75 (2022) 261–321. doi:10.1613/jair.1.13575.

[37] C. Zhang, H. Zhang, C.-J. Hsieh, An efficient adversarial attack for tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 16165–16176.

[38] H. Chen, H. Zhang, S. Si, Y. Li, D. Boning, C.-J. Hsieh, Robust decision trees against adversarial examples, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019, pp. 1122–1131.

[39] M. Andriushchenko, M. Hein, Provably robust boosted decision stumps and trees against adversarial attacks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 12997–13008.

[40] S. Calzavara, C. Lucchese, G. Tolomei, S. A. Abebe, S. Orlando, Treant: training evasion-aware decision trees, Data Mining and Knowledge Discovery 34 (5) (2020) 1390–1420. doi:10.1007/s10618-020-00694-9.

[41] Z. Lin, Y. Shi, Z. Xue, IDSGAN: Generative adversarial networks for attack generation against intrusion detection, arXiv preprint arXiv:1809.02077 (2018). doi:10.48550/arXiv.1809.02077.

[42] M. J. Hashemi, E. Keller, Enhancing robustness against adversarial examples in network intrusion detection systems, in: Proceedings of the 2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), 2020, pp. 37–43. doi:10.1109/NFV-SDN50289.2020.9289869.

[43] J. Postel, Internet protocol, RFC 791, RFC Editor (1981). doi:10.17487/RFC0791.

[44] R. Braden, D. Borman, C. Partridge, Computing the internet checksum, RFC 1071, RFC Editor (1988). doi:10.17487/RFC1071.

[45] W. Eddy, Transmission control protocol (TCP) specification, RFC 9293, RFC Editor (2022). doi:10.17487/RFC9293.

[46] J. Postel, User datagram protocol, RFC 768, RFC Editor (1980). doi:10.17487/RFC0768.

[47] C. Hornig, A standard for the transmission of IP datagrams over Ethernet networks, RFC 894, RFC Editor (1984). doi:10.17487/RFC0894.

[48] T. Ylonen, C. Lonvick, The secure shell (SSH) transport layer protocol, RFC 4253, RFC Editor (2006). doi:10.17487/RFC4253.

[49] M. Allman, V. Paxson, E. Blanton, TCP congestion control, RFC 5681, RFC Editor (2009). doi:10.17487/RFC5681.

[50] D. Borman, B. Braden, V. Jacobson, R. Scheffenegger, TCP extensions for high performance, RFC 7323, RFC Editor (2014). doi:10.17487/RFC7323.

[51] C. Carathéodory, Vorlesungen über reelle Funktionen, B. G. Teubner, Leipzig, 1918.

[52] M. Nagumo, Über die Lage der Integralkurven gewöhnlicher Differentialgleichungen, Proceedings of the Physico-Mathematical Society of Japan. 3rd Series 24 (1942) 551–559. doi:10.11429/ppmsj1919.24.0_551.

[53] H. Brézis, On a characterization of flow-invariant sets, Communications on Pure and Applied Mathematics 23 (2) (1970) 261–263. doi:10.1002/cpa.3160230211.

[54] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[55] V. Paxson, M. Allman, J. Chu, M. Sargent, Computing TCP's retransmission timer, RFC 6298, RFC Editor (2011). doi:10.17487/RFC6298.

[56] M. Thomson, C. Benfield, HTTP/2, RFC 9113, RFC Editor (2022). doi:10.17487/RFC9113.

[57] J. Postel, J. Reynolds, File transfer protocol (FTP), RFC 959, RFC Editor (1985). doi:10.17487/RFC0959.